

1 GREENFAAS: Carbon-Aware Scheduling for Serverless 2 Workloads

3 Abdeldjalil Ledmi^{a,*}, Makhlouf Ledmi^a, Mohammed El Habib Souidi^a,
4 Nabil Azizi^a, Abdelkarim Oucif^a, Ferial Laassami^a, Hichem Haouassi^a,
5 Toufik Messaoud Maarouk^a

*^aDepartment of Computer Science, ICOSI Lab, Abbes Laghrou University of
Khenchela, Khenchela, 40000, Algeria*

6 Abstract

7 Data centres' growing electricity footprint has motivated a body of research
8 on carbon-aware scheduling, in which workloads are shifted in time or space
9 to coincide with cleaner electricity. Much of the foundational carbon-aware
10 scheduling literature, however, targets batch or long-running jobs, while re-
11 cent FaaS-specific systems each address only part of the spatial, temporal,
12 hardware, and autoscaling design space. The assumptions of the batch-
13 oriented systems — minutes-to-hours runtime, delay tolerance, and clean
14 suspend/resume — are profoundly violated by serverless or Function-as-a-
15 Service (FaaS) workloads. FaaS functions execute in milliseconds to seconds,
16 often under sub-second SLAs, suffer expensive cold starts when interrupted,
17 and arrive in bursty, event-driven patterns.

18 We present GREENFAAS, a carbon-aware scheduling framework for server-
19 less workloads that unifies three decisions previously treated separately: spa-
20 tial routing across regions, temporal deferral within SLA bounds, and warm-
21 pool-aware container reuse. Four prior FaaS-targeted carbon-aware sched-
22 ulers cover different facets of the design space: *GreenCourier* (Chadha et al.,

*Corresponding author.

Email addresses: `abdeldjalil.ledmi@univ-khenchela.dz` (Abdeldjalil Ledmi),
`ledmi.makhlouf@univ-khenchela.dz` (Makhlouf Ledmi),
`souidi.mohammed@univ-khenchela.dz` (Mohammed El Habib Souidi),
`azizi.nabil@univ-khenchela.dz` (Nabil Azizi),
`oucif.abdelkarim@univ-khenchela.dz` (Abdelkarim Oucif),
`ferial.laassami@univ-khenchela.dz` (Ferial Laassami),
`haouassi.hichem@univ-khenchela.dz` (Hichem Haouassi),
`maarouk.toufik@univ-khenchela.dz` (Toufik Messaoud Maarouk)

23 2023) routes single functions to low-carbon regions; *Caribou* (Gsteiger et al.,
 24 2024) shifts whole serverless workflows geographically; *EcoLife* (Jiang et al.,
 25 2024) exploits multi-generation hardware; and *CASA* (Qi et al., 2024) co-
 26 optimizes autoscaling and SLOs within a single cluster. GREENFAAS is the
 27 first to give a closed-form joint analysis of the spatial and temporal axes un-
 28 der cold-start economics, and to demonstrate empirically-validated robust-
 29 ness across topology and workload regimes.

30 The paper’s principal analytical contribution is a closed-form *cold-start*
 31 *carbon trade-off*: we prove a dichotomy in which the warm-pool placement
 32 decision is governed by a dimensionless idle-cost ratio $\beta = P_i T_w / (P_a \tau_e)$ rel-
 33 ative to a critical threshold $(r - 1)(1 + \alpha)$, with a closed-form break-even
 34 condition and a unique critical arrival rate λ^* — the root of a monotone
 35 scalar equation — separating the regimes. The same idle-cost ratio governs
 36 the analogous temporal-shift trade-off with the deferral interval Δt in place
 37 of T_w , giving the scheduler principled decision rules along both axes.

We evaluate GREENFAAS in a discrete-event simulator, driven by a schema-
 faithful synthetic workload calibrated to the Azure Functions 2019 and 2021
 characterizations and by real grid carbon-intensity data from Let’s-Wait-
 Awhile (Wiesner et al., 2021; dos-group, 2021) and from ElectricityMaps
 over the same January 2021 window as the Azure trace, against four carbon-
 aware baselines representative of recent work. Three findings stand out.
First, GREENFAAS reduces operational carbon by 68–83% relative to FIFO
 across a range of topologies on fully-real time-aligned data (real Azure 2021
 trace + real ElectricityMaps Jan 2021 carbon), and 64–72% on synthetic-
 workload setups. On fully-real time-aligned data the GREENFAAS–Spatial

gap is 0.04 percentage points; across all topologies on real-carbon data the gap stays within 0.5pp. GREENFAAS’s distinct value over Spatial is topology-robustness rather than dominance in any single regime. *Second*, GREENFAAS preserves a *do-no-harm* property that extends beyond zero-opportunity workloads to a robust neighbourhood of low-opportunity ones: across shiftable fractions $\{0, 1\%, 2\%\}$ and in single-region scenarios on both synthetic and fully-real data, GREENFAAS matches FIFO byte-for-byte; a heuristic ablation lacking the cold-start trade-off correction loses 5–102% to FIFO in the same scenarios. *Third*, the canonical Wait-Awhile temporal scheduler and a FaaS-adapted port of the provably-optimal one-way trading algorithm of Lechowicz et al. (2023) both *fail* on FaaS workloads, losing 3–306% to FIFO depending on the regime — catastrophically (up to -306%) at batch-default deferral horizons on synthetic workloads, and mildly (3–5%) on fully-real data where they degenerate toward a FIFO no-op. In neither regime do they reduce carbon: a clean failure mode for batch carbon-aware techniques on FaaS, including those with provable competitive guarantees, when warm-pool idle-energy coupling is not accounted for. Head-to-head against the closest competing serverless system, *Caribou* (Gsteiger et al., 2024), well-tuned Caribou (hourly re-deployment with perfect carbon forecasting) and GREENFAAS achieve statistically indistinguishable carbon savings (within 0.1pp) on real Azure 2021 workload; the difference is operational rather than empirical, with GREENFAAS avoiding the container migration and pub/sub orchestration that Caribou requires. The simulator, scheduler, trace loaders, Caribou port, and real Azure Functions and ElectricityMaps integrations are released as open-source artifacts under the Apache-2.0 license upon publication.

38 *Keywords:* carbon-aware scheduling, serverless, Function-as-a-Service, cold
39 start, sustainable computing

40 1. Introduction

41 Data centres consume an increasing share of global electricity, and hyper-
42 scale operators have made public commitments to carbon-neutral or net-zero
43 operation within the decade. These pledges have motivated substantial re-
44 search on *carbon-aware computing*: techniques that exploit spatial and tem-
45 poral variation in grid carbon intensity to shift computation toward cleaner
46 energy. The dominant strategy in the literature is *temporal shifting* of delay-
47 tolerant work to periods of low carbon intensity (Wiesner et al., 2021); com-
48plementary *spatial shifting* routes work toward regions with cleaner grids
49 (Sukprasert et al., 2024; Radovanović et al., 2023). Both have delivered 20–
50 45% operational-carbon reductions for batch processing, machine learning
51 training, and similar long-running workloads (Hanafy et al., 2023; Sukprasert
52 et al., 2024).

53 A critical assumption underlies essentially this entire body of work: the
54 target workload is *batch* in character. The canonical scheduled job is min-
55 utes to hours long, delay-tolerant on the order of hours, amenable to clean
56 suspend-and-resume, and accompanied by enough up-front information that
57 the scheduler can reason globally over a set of pending jobs. These assump-
58 tions are violated profoundly by *serverless* or *Function-as-a-Service* (FaaS)
59 workloads, which now power a substantial fraction of cloud-native applica-
60 tions. FaaS invocations are typically measured in tens of milliseconds to a few
61 seconds; they are often user-facing with sub-second latency objectives; they

cannot be suspended and resumed without paying the full cost of a *cold start*; their arrivals are bursty and event-driven; and per-invocation behaviour is largely unpredictable until execution.

This mismatch is consequential. Recent FaaS-targeted carbon-aware schedulers — *GreenCourier* (Chadha et al., 2023), *Caribou* (Gsteiger et al., 2024), *EcoLife* (Jiang et al., 2024), and *CASA* (Qi et al., 2024) — have each begun to address the gap from a distinct angle: spatial routing of single functions (GreenCourier), workflow-level geo-shifting (Caribou), hardware-generation choice (EcoLife), and SLO/carbon-aware autoscaling (CASA). None of them, however, formalizes the joint spatial/temporal/warm-pool decision problem that FaaS schedulers actually face, and the fundamental cold-start versus idle-energy trade-off has not previously been characterized in closed form.

We argue that carbon-aware scheduling for FaaS is both feasible and distinct enough from its batch counterpart to warrant first-class treatment. Three observations motivate our approach. *First*, FaaS workloads are heterogeneous in latency tolerance: a customer-facing API request demands sub-second response, but many event-driven functions tolerate seconds to minutes of deferral without observable user impact. *Second*, FaaS platforms already perform fine-grained placement and routing decisions for load-balancing and cold-start mitigation; carbon-aware policies can be layered onto this existing machinery. *Third*, warm-pool management — how many idle containers to keep alive in each region — is itself a carbon decision, because idle containers consume non-trivial power and the choice of where to keep them warm interacts directly with the carbon intensity of the host grid.

In this work, we present GREENFAAS, a carbon-aware scheduling frame-

work for serverless workloads that unifies spatial routing, temporal deferral, and warm-pool-aware container reuse under a single principled policy. GREENFAAS combines temporal deferral for asynchronous and event-driven invocations, spatial routing for latency-tolerant classes, and carbon-aware warm-pool management, all under an SLA-tiered policy distinguishing interactive, deferrable, and background functions. A key technical contribution is the explicit modelling of the *cold-start carbon trade-off*: keeping containers warm in a low-carbon region can, in some regimes, cost more carbon (through idle power) than cold-starting in a higher-carbon region. We derive analytical break-even points for this trade-off and incorporate them directly into the scheduler.

We evaluate GREENFAAS in **GreenFaaS-sim**, a discrete-event simulator whose design follows the abstractions of **faas-sim** and **vessim** (Edgerun, 2022; dos-group, 2023), driven by the real Azure Functions 2021 per-invocation trace (Zhang et al., 2021; Microsoft, 2021) and by real grid carbon traces from Let’s-Wait-Awhile (Wiesner et al., 2021; dos-group, 2021) and from ElectricityMaps over the same January 2021 window as the Azure trace. The trace loaders also accept a schema-faithful synthetic workload as a fallback when the real trace is not locally available (§8.1). We compare against four carbon-aware baselines drawn from the recent literature, evaluating along five axes: workload intensity, number of available regions, SLA class mix, carbon-intensity variability, and forecast accuracy.

Our key findings are threefold. *First*, GREENFAAS reduces operational carbon by 68–83% across topologies on fully-real time-aligned data (Azure 2021 trace + ElectricityMaps Jan 2021 carbon), and by 64–72% on synthetic-

workload setups; on fully-real data the GREENFAAS–Spatial gap is just 0.04 percentage points. *Second*, GREENFAAS preserves a *do-no-harm* property: in single-region and flat-carbon regimes where shifting offers no real opportunity, GREENFAAS matches FIFO byte-for-byte on both synthetic and fully-real data, while a heuristic ablation (lacking the cold-start trade-off correction) loses 5–102% to FIFO in the same scenarios. *Third*, the canonical Wait-Awhile temporal scheduler and a FaaS-adapted port of the Lechowicz et al. (2023) provably-optimal one-way trading algorithm both *fail* on FaaS workloads (3–306% over FIFO depending on the regime), an empirically sharp demonstration that batch-oriented carbon-aware techniques — including those with provable competitive guarantees — do not transfer to FaaS without addressing warm-pool idle-energy coupling. This robustness across regimes — rather than dominance in any single regime — is the central practical claim of the paper.

Contributions.. The paper makes the following contributions:

1. A formal model of carbon-aware FaaS scheduling as an online optimization problem with capacity, SLA, and warm-pool constraints, and an honest discussion of why classical competitive-ratio bounds do not apply (§4).
2. A closed-form *cold-start carbon trade-off lemma* characterizing warm-pool versus cold-start carbon as a function of arrival rate, runtime and cold-start durations, active and idle power, warm-pool TTL, and the carbon-intensity ratio between regions (§5). The same dimensionless idle-cost ratio governs the analogous temporal-shift decision via an idle-energy correction (§5.4).

- 137 3. The GREENFAAS scheduler: a hybrid online algorithm combining tem-
 138 poral deferral, spatial routing, and lemma-driven warm-pool manage-
 139 ment under an SLA-tiered policy, with $O(R \cdot N)$ per-invocation cost
 140 (§6).
- 141 4. An open-source simulator with trace loaders that accept the published
 142 Azure Functions 2019, Azure Functions 2021, and LWA carbon-intensity
 143 schemas without modification (§7).
- 144 5. An empirical evaluation against four carbon-aware baselines and a
 145 five-axis sensitivity sweep establishing the do-no-harm property across
 146 topology, SLA-class mix, forecast accuracy, carbon variability, and
 147 workload intensity (§8).

148 The remainder of this paper is organized as follows. §2 surveys related
 149 work; §3 motivates the problem; §4 formalizes it; §5 derives the trade-off
 150 lemma; §6 presents the algorithm; §7 describes the simulator; §8 reports
 151 results; §11 discusses limitations; §12 concludes.

152 2. Related Work

153 Carbon-aware scheduling has matured rapidly over the past four years.
 154 We organise prior work into four threads: (i) batch and long-running work-
 155 loads, where the dominant approaches assume delay-tolerance and suspend/resume;
 156 (ii) theoretical foundations of online carbon-aware scheduling; (iii) the small
 157 but growing set of carbon-aware schedulers explicitly targeting serverless or
 158 FaaS workloads, with which GREENFAAS most directly engages; and (iv)
 159 FaaS systems work on cold-starts, warm pools, and characterization, on
 160 which our cost model depends.

161 2.1. Carbon-Aware Scheduling for Batch Workloads

162 Wiesner et al.’s *Let’s Wait Awhile* (Wiesner et al., 2021) introduced
 163 threshold-based deferral for batch workloads and released the hourly carbon-
 164 intensity dataset (DE, GB, FR, US-CAISO, 2020) that has become the de-
 165 facto reference; we use this dataset directly in our evaluation (§8.1). *Car-*
 166 *bonScaler* (Hanafy et al., 2023) scales the parallelism of ML training jobs in
 167 response to carbon intensity, exploiting the elasticity of data-parallel work-
 168 loads. Souza et al.’s *GAlIA* scheduler (Sukprasert et al., 2024) addresses
 169 the three-way trade-off between carbon, performance, and cost for batch
 170 jobs, and quantifies the practical limits of carbon-aware spatiotemporal shift-
 171 ing; the same group’s *Ecovisor* (Souza et al., 2023) provides a virtual en-
 172 ergy system abstraction. Google’s Carbon-Intelligent Compute (Radovanović
 173 et al., 2023) deploys a Variable Capacity Curve approach at production scale.
 174 *LACS* (Bostandoost et al., 2024) is a learning-augmented resource scaler ad-
 175 dressing uncertain demand. *Adapting Datacenter Capacity* (Lin and Chien,
 176 2023) explored provisioning to track grid carbon.

177 These works share two structural assumptions that GREENFAAS relaxes.
 178 *First*, the schedulable units are *jobs* with cost separable per-job, allowing
 179 the schedulers to reason about each job’s deferral in isolation. FaaS invoca-
 180 tions are not separable in this sense because their cost is coupled through
 181 warm-pool state, which we formalize in §4. *Second*, the schedulers operate
 182 on a timescale (minutes to hours) where cold-start penalties are negligible.
 183 We show empirically in §8 that naively transplanting these techniques to
 184 FaaS workloads — as the Wait-Awhile baseline does — *more than doubles*
 185 operational carbon relative to a carbon-unaware FIFO baseline (163–236%

186 above FIFO across our synthetic and real-carbon experiments). The “batch
187 schedulers don’t transfer” claim is therefore not only intuitively obvious from
188 the assumption mismatch but also empirically severe.

189 2.2. Theoretical Foundations

190 A recent line of work has put carbon-aware online algorithms on firmer
191 footing. Lechowicz et al.’s *Online Pause and Resume Problem* (Lechowicz
192 et al., 2023) studies a one-dimensional model in which a workload can be
193 paused during high-carbon periods and resumed during low-carbon periods,
194 paying a fixed switching cost. They derive double-threshold algorithms with
195 provably optimal competitive ratios. These theoretical results apply to a
196 workload model that is in several respects strictly simpler than ours: a single
197 workload (no multi-tenant mix), one machine (no spatial routing), and a sin-
198 gle switching cost (no idle-energy accumulation). The FaaS setting we study
199 introduces three structural features — per-invocation cold-start versus warm-
200 pool trade-offs, spatial routing across regions, and tiered SLA constraints —
201 that take us outside the class of problems for which competitive-ratio bounds
202 are currently known. We are explicit in §4.2 that GREENFAAS is an empirical
203 scheduler with a *local* analytical guarantee (Lemma 5.1’s break-even charac-
204 terization) rather than a competitively-optimal one. Extending (Lechowicz
205 et al., 2023) to handle warm-pool idle energy and SLA tiering is an open
206 direction we identify in §11.

207 2.3. Carbon-Aware FaaS and Serverless

208 Five recent systems target serverless workloads specifically; each covers a
209 different facet of the design space.

210 *GreenCourier* (Chadha et al., 2023).. The closest single-function spatial-
 211 routing precedent: a Kubernetes scheduler plugin that routes serverless func-
 212 tions across geographically distributed Knative clusters based on real-time
 213 grid carbon intensity. It reports 9–18% carbon reduction per invocation
 214 with negligible latency overhead. GreenCourier and GREENFAAS share the
 215 high-level goal but differ in three significant respects. *First*, GreenCourier is
 216 *spatial-only* — it routes functions to the cleanest available region, with no
 217 temporal deferral and no warm-pool reasoning. Our headline contribution be-
 218 yond GreenCourier is therefore the joint spatial/temporal/warm-pool policy
 219 and the underlying trade-off analysis. *Second*, GreenCourier has no analytical
 220 model of the warm-pool versus cold-start carbon trade-off; our Lemma 5.1
 221 gives a closed-form characterization that becomes essential as soon as the
 222 scheduler reasons about *whether* to maintain a warm pool. *Third*, Green-
 223 Courier’s reported savings (9–18%) sit at the lower end of what our headline
 224 results suggest is achievable; we attribute this to the spatial-only scope rather
 225 than to algorithmic deficiencies in GreenCourier itself.

226 *Caribou* (Gsteiger et al., 2024).. Caribou is the closest recent system in scope
 227 at the workflow level. It offloads serverless *workflows* (multi-function DAGs)
 228 across geo-distributed regions for sustainability, automatically (re-)deploying
 229 functions and redirecting traffic to lower-carbon endpoints. Caribou requires
 230 no changes to application logic and preserves performance and cost metrics.
 231 Its scope is workflow-level spatial (no temporal deferral, no per-invocation
 232 warm-pool reasoning), with decisions made at deployment time and refreshed
 233 by periodic redeployment rather than per invocation.

234 We compare head-to-head in §8.7 by porting Caribou’s deployment-solver

235 logic to our per-invocation workload as a single-node DAG case, with per-
 236 fect carbon forecasting and constant token-bucket grant (both favourable to
 237 Caribou). The finding: Caribou(1h) and GREENFAAS achieve carbon sav-
 238 ings that differ by less than 0.1 percentage points on our 24h FaaS slices; the
 239 two systems are operationally distinct but empirically equivalent on per-
 240 invocation workloads with hour-scale carbon variation. The systems are
 241 complementary along the granularity axis: Caribou makes coarse-grained,
 242 deployment-time placement decisions for whole workflows; GREENFAAS makes
 243 fine-grained, per-invocation decisions including warm-pool reasoning. A nat-
 244 ural composition would use Caribou for workflow placement and GREEN-
 245 FAAS for per-invocation scheduling within each chosen region; we have not
 246 implemented this composition.

247 *EcoLife* (Jiang et al., 2024).. EcoLife targets the hardware-generation axis:
 248 it co-optimizes *operational* and *embodied* carbon by exploiting multi-generation
 249 hardware (newer servers are faster but have higher embodied carbon per
 250 unit time; older servers are slower but already amortised). It uses a Parti-
 251 cle Swarm Optimisation-based scheduler that decides, per invocation, which
 252 hardware generation to execute on. EcoLife explicitly reasons about keep-
 253 alive (warm-pool) versus cold-start trade-offs — which makes it the closest
 254 precedent for our Lemma 5.1 — but it does so through a metaheuristic op-
 255 timizer without a closed-form characterization. The two contributions are
 256 largely orthogonal: EcoLife answers “on what hardware generation should
 257 this function execute?”; GREENFAAS answers “in what region and at what
 258 time should this function execute, and is a warm pool worth maintaining
 259 there?”. A scheduler combining the two axes is a natural follow-up (§11).

260 *CASA* (Qi et al., 2024).. *CASA* is the closest recent single-cluster, autoscaling-
 261 focused competitor. It is an SLO- and carbon-aware framework that sched-
 262 ules and autoscales containers in a serverless cluster, explicitly trading off
 263 the SLO cost of cold-starts against the carbon cost of pre-warming and keep-
 264 alive. *CASA* reports up to a $2.6\times$ reduction in operational carbon while
 265 reducing SLO violations by up to $1.4\times$ over a state-of-the-art baseline. The
 266 follow-on work (Qi et al., 2024) extends the same multi-objective framework
 267 to co-optimize wastewater alongside carbon and SLOs. *CASA* and *GREEN-*
 268 *FAAS* address overlapping concerns through different mechanisms. *CASA*’s
 269 contribution is a carbon-aware autoscaling and container-placement policy
 270 in a single cluster, evaluated by simulation against a state-of-the-art au-
 271 toscaler. *GREENFAAS*’s contribution is a closed-form trade-off characteriza-
 272 tion (Lemma 5.1) that yields a parameter-free per-invocation decision rule,
 273 combined with a multi-region routing policy and a formal do-no-harm prop-
 274 erty (§8.8, §8.9). Where *CASA* establishes that carbon-aware autoscaling is
 275 worthwhile, *GREENFAAS* establishes the analytical conditions under which
 276 warm-pool maintenance is itself carbon-positive — a question *CASA*’s heuris-
 277 tics do not pose explicitly. The two approaches could be composed at different
 278 granularities: *CASA* at the container-pool autoscaling layer, *GREENFAAS* at
 279 the per-invocation routing-and-warm-pool-gate layer.

280 *GreenWhisk* (Serenari et al., 2024).. *GreenWhisk* is a system-level redesign
 281 of Apache OpenWhisk to expose energy and carbon state to load-balancing
 282 algorithms, supporting both grid-connected and grid-isolated modes. Its con-
 283 tribution is infrastructural: an energy-interface API and a carbon-aware load-
 284 balancer integrated into a production-grade FaaS platform. The scheduling

algorithm itself is intentionally minimal in scope; the value is the open-platform abstraction. GREENFAAS’s algorithmic and analytical contributions are compatible with this abstraction: a production GreenWhisk deployment could load Algorithm 1 as its load-balancing policy without changes to either side. GreenWhisk’s two-mode design (grid intermittency) is a deployment dimension our analysis does not address and that an integration would have to handle separately.

CASPER (Souza et al., 2024).. Adjacent rather than directly competing: targets distributed web services (MediaWiki-class applications) under latency SLOs, using a mixed-integer program to jointly provision servers and load-balance requests across cloud regions. It addresses spatial routing and provisioning with SLO constraints but treats requests at the application level (request rates, not per-invocation arrivals) and does not engage with cold-start economics. We position GREENFAAS as the per-invocation, FaaS-native analogue of CASPER, with the cold-start trade-off model as a key technical distinction.

2.4. Serverless Cold-Starts and Characterisation

Our cost model in §4 and the SLA-tiered policy in §6.3 depend on a substantial body of FaaS systems work. *Serverless in the Wild* (Shahrad et al., 2020) is the canonical characterization of production Azure Functions workloads; it established that invocations are dominated by a small number of high-frequency “hot” functions, that durations follow heavy-tailed distributions, and that arrival processes are bursty and event-driven. Our workload generator and the function popularity model (Zipf with $s = 1.2$) follow this

309 characterization directly. The follow-up *Azure Functions Invocation Trace*
 310 *2021* (Zhang et al., 2021; Microsoft, 2021) gives per-invocation records over
 311 two weeks; we use its schema as the basis for our trace loader (§8.1).

312 The cold-start problem itself has a rich literature. *Catalyzer* (Du et al.,
 313 2020) and *Firecracker* (Agache et al., 2020) attack cold-starts at the sys-
 314 tem level via micro-VM and snapshot techniques. *FaaS-Cache* (Fuerst and
 315 Sharma, 2021) frames warm-pool management as a caching problem and de-
 316 rives keep-alive policies from caching theory. We borrow the warm-pool /
 317 keep-alive abstraction directly, but add the carbon dimension: idle energy
 318 at intensity C_r in region r is itself a cost that the scheduler must trade off
 319 against cold-start energy at potentially different C . This is the substance of
 320 our Lemma 5.1.

321 A parallel measurement-focused line of work makes it possible to attribute
 322 per-invocation carbon footprints to FaaS workloads with high accuracy. *Ac-*
 323 *countable Carbon Footprints* (Sharma and Fuerst, 2024) develops a Shapley-
 324 value-based disaggregation methodology that fairly attributes shared hard-
 325 ware and software emissions to individual function invocations, achieving
 326 >99% accuracy on a wide range of workloads. This work is methodologically
 327 orthogonal — it measures, we schedule — but a production-grade carbon-
 328 aware scheduler should integrate with such a measurement substrate to close
 329 the loop from decisions to observed outcomes.

330 *Summary of positioning.* To our knowledge, no prior work has formalized
 331 the cold-start carbon trade-off in closed form, demonstrated the temporal-
 332 axis idle-energy correction (§5.4), or established a do-no-harm property across
 333 topology, SLA-mix, forecast-accuracy, and carbon-variability axes. These are

334 the empirical and analytical novelties on which the rest of the paper builds.

335 3. Motivation

336 We motivate carbon-aware FaaS scheduling with a joint characterization
337 of the two data sources that drive our design: a representative FaaS workload,
338 and the grid carbon-intensity signal that determines the carbon footprint of
339 executing that workload.

340 3.1. Joint Characterisation

341 Figure 1 overlays a 48-hour grid carbon-intensity trace for Germany —
342 a synthetic trace whose annual mean is calibrated to the ENTSO-E 2020
343 figure later validated against real LWA data in §8.3 — with the per-minute
344 invocation rate of a representative `webhook_handler` function drawn from a
345 workload generator calibrated to the Shahrad et al. (2020) Azure character-
346 ization. Shaded bands mark the lowest-quartile (green) and highest-quartile
347 (red) carbon windows over the 48-hour horizon. Three observations emerge.

348 1. *FaaS arrival rates are decoupled from grid carbon intensity.* The webhook
349 function’s rate follows a business-hours diurnal pattern, peaking in early af-
350 ternoon and troughing overnight. Grid carbon intensity in Germany follows
351 the inverse pattern: solar and wind generation peak around midday, depress-
352 ing the marginal intensity, while night-time demand is served by base-load
353 fossil generation. The two signals are phase-shifted by roughly 8–12 hours; on
354 this representative day, 68% of webhook invocations land *outside* the cleanest
355 carbon quartile and 18% land directly in the highest-carbon quartile. The
356 decoupling is the structural reason carbon-aware scheduling can help.

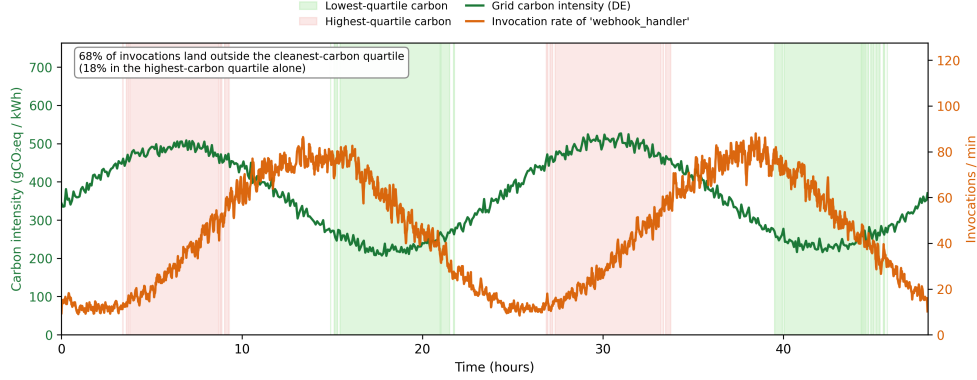


Figure 1: FaaS arrivals are decoupled from grid carbon intensity. The orange line (right axis) shows the per-minute invocation rate of the `webhook_handler` function; the green line (left axis) shows grid carbon intensity in Germany. The two signals are phase-shifted by 8–12 hours, creating temporal shifting opportunity for deferrable invocations. Shaded bands mark the lowest- (green) and highest-carbon (red) quartile windows.

357 2. *The decoupling is robust across function classes.* Across the 60-function
 358 catalog generated for our evaluation, the population-weighted invocation rate
 359 correlates only weakly with the local carbon intensity (Pearson r in the range
 360 -0.2 to $+0.1$ across regions), confirming that arrival patterns track user
 361 activity rather than grid economics — as expected, since FaaS workloads are
 362 driven by external events (user requests, message-queue depths, scheduled
 363 triggers) that have no relationship to the regional generation mix.

364 3. *The amplitude of the carbon swing is comparable to the amplitude of*
 365 *the arrival swing.* Carbon intensity in Germany swings from ~ 200 to ~ 500
 366 $\text{gCO}_2\text{eq/kWh}$ over the 24-hour cycle — a ratio of $2.5\times$. The webhook in-
 367 vocation rate swings from ~ 15 to ~ 80 invocations per minute — a ratio of
 368 $5\times$. The two signals have similar dynamic range, which means a scheduler

369 that can shift even a modest fraction of invocations across the diurnal cy-
370 cle has potential to capture meaningful savings. We quantify this potential
371 rigorously in §8.

372 3.2. Implications for Scheduler Design

373 The characterization directly motivates three design choices in §6. *First*,
374 the SLA-tiered policy: arrivals distributed across the carbon cycle are only
375 useful to a scheduler that distinguishes between invocations that must exe-
376 cute immediately (interactive) and those that can wait. *Second*, the use of
377 *both* spatial and temporal axes: a purely temporal scheduler can only exploit
378 the diurnal swing within one region; a purely spatial scheduler ignores the
379 substantial within-region opportunity Figure 1 reveals. *Third*, the explicit
380 carbon-aware warm-pool reasoning of §5: at the timescale of FaaS invoca-
381 tions (seconds), the idle-energy cost of keeping containers warm is a first-class
382 concern.

383 The motivation, in short, is not “carbon-aware scheduling helps FaaS”
384 in some generic sense — that has been shown by the four FaaS-targeted
385 systems surveyed in §2.3 (Chadha et al., 2023; Gsteiger et al., 2024; Jiang
386 et al., 2024; Qi et al., 2024). The motivation is that the *specific structure*
387 of FaaS workloads (short, bursty, latency-tiered) combined with the *specific*
388 *structure* of grid carbon traces (diurnal, decoupled from user activity, multi-
389 region heterogeneous) creates a joint optimization problem that no prior
390 scheduler has addressed in its full generality. The rest of the paper formalizes
391 and solves that problem.

392 4. Problem Formulation

393 This section formalizes the carbon-aware FaaS scheduling problem. §4.1
 394 sets up the entities, decision variables, and constraints. §4.2 frames the opti-
 395 mization as an online problem and discusses why it admits neither a tractable
 396 offline optimum nor a competitive-ratio bound, motivating the empirical eval-
 397 uation strategy in §8.

398 4.1. Problem Statement

399 *Regions..* A set $\mathcal{R} = \{r_1, \dots, r_R\}$ of geographic regions. Each region r
 400 has capacity K_r , a time-varying grid carbon intensity $C_r : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{>0}$ in
 401 gCO₂eq/kWh, a Power Usage Effectiveness $\text{PUE}_r \geq 1$, and network round-
 402 trip latencies $\rho(r, r') \geq 0$ to every other region.

403 *Functions..* A set $\mathcal{F}$ of deployed functions, each $f \in \mathcal{F}$ characterized by ex-
 404 pected execution time $\tau_e(f)$, cold-start duration $\tau_c(f)$, active power $P_a(f)$,
 405 idle power $P_i(f)$, latency class $\ell(f) \in \{\text{INTERACTIVE}, \text{DEFERRABLE}, \text{BACKGROUND}\}$,
 406 and SLA deadline $D(f)$. The latency class determines the scheduler’s de-
 407 gree of freedom: INTERACTIVE invocations must execute immediately in
 408 their home region; DEFERRABLE invocations may be deferred up to a class-
 409 dependent horizon and routed within a tight RTT budget; BACKGROUND
 410 invocations may use their full deadline and any region.

411 *Invocations..* Function invocations arrive as a stream $\mathcal{I} = \langle i_1, i_2, \dots \rangle$ ordered
 412 by arrival time. Each i specifies a function $f(i)$, an arrival time $a(i)$, a home
 413 region $h(i)$, and a realised runtime $\tau(i)$ drawn from $f(i)$ ’s execution-time
 414 distribution. Crucially, $\tau(i)$ is *not* known to the scheduler at decision time;

only the expected runtime $\tau_e(f(i))$ is. The same applies to future arrivals:
 at time $a(i)$ the scheduler sees i but knows nothing about i_k for k with
 $a(i_k) > a(i)$ beyond what an arrival-rate estimator can infer.

Warm pools.. For each $(r, f) \in \mathcal{R} \times \mathcal{F}$, the platform maintains a non-negative
 integer warm-container count $W_{r,f}(t)$ representing containers that have pre-
 viously executed f in r and remain alive awaiting reuse. A warm container
 is evicted after a TTL of T_w seconds of inactivity.

Decisions.. For each invocation i , the scheduler produces $\pi(i) = (r(i), s(i))$:
 the execution region $r(i)$ and the scheduled start time $s(i)$. The secondary
 outcome $w(i) = \mathbf{1}[W_{r(i),f(i)}(s(i)) > 0]$ — whether i finds a warm container at
 $r(i)$ at time $s(i)$ — is observed rather than chosen; if $w(i) = 0$, the invocation
 pays $\tau_c(f(i))$ in start-up latency and energy.

Per-invocation cost.. The carbon cost of i under $\pi(i) = (r, s)$ is

$$\text{Carbon}(i) = \frac{\text{PUE}_r}{3.6 \times 10^6} C_r(s) \cdot P_a(f(i)) [\tau(i) + (1 - w(i))\tau_c(f(i))].$$

End-to-end latency is

$$\text{Lat}(i) = (s - a(i)) + \rho(h(i), r) + (1 - w(i))\tau_c(f(i)) + \tau(i).$$

System cost.. Aggregate carbon over horizon $[0, T]$ sums per-invocation exe-
 cution carbon and warm-pool idle carbon. The exact idle carbon integrates
 the time-varying intensity over each idle interval:

$$\begin{aligned} \text{TotalCarbon}([0, T]) &= \sum_{i: a(i) \in [0, T]} \text{Carbon}(i) \\ &\quad + \sum_{r, f} \frac{\text{PUE}_r}{3.6 \times 10^6} P_i(f) \int_0^T W_{r,f}(t) C_r(t) dt, \end{aligned} \quad (1)$$

432 where $W_{r,f}(t)$ is the number of warm-but-idle containers for function f in
 433 region r at time t , and $C_r(t)$ is the instantaneous carbon intensity. For
 434 analytical tractability in Lemma 5.1 and in the SCORESLOTS heuristic, we
 435 approximate $C_r(t)$ over an idle interval by its mean $\bar{C}_r$. This approximation
 436 is exact when warm-pool idle time is distributed uniformly over the horizon
 437 (the typical case for steady FaaS load, where the warm pool is continuously
 438 cycled); deviation arises only if idle time correlates with carbon intensity.
 439 Because GreenFaaS’s spatial routing concentrates load in low-carbon regions
 440 during high-carbon periods, any residual correlation is conservative (it would
 441 slightly *over*-estimate idle carbon in the regions GreenFaaS avoids). The
 442 simulator (§7) reports results under this mean-intensity approximation; the
 443 scheduler *ranking* is what the paper claims, and the power-model robustness
 444 check (§11) confirms the ranking is insensitive to the idle-energy magnitude.

445 *Constraints..* A schedule π must respect: (1) *Capacity*: at every t , the count
 446 of in-flight invocations in region r is at most K_r . (2) *SLA-feasibility in expectation*:
 447 $(s(i) - a(i)) + \rho(h(i), r(i)) + (1 - \hat{w}(i))\tau_c(f(i)) + \tau_e(f(i)) \leq D(f(i))$,
 448 where $\hat{w}(i)$ is the scheduler’s warm-pool prediction. (3) *Class-dependent shifting limits*:
 449 INTERACTIVE $\Rightarrow s(i) = a(i)$ and $r(i) = h(i)$; DEFERRABLE
 450 $\Rightarrow s(i) \leq a(i) + \Delta^D$ and $\rho(h(i), r(i)) \leq \rho^D$; BACKGROUND $\Rightarrow$ analogous with
 451 class-specific bounds Δ^B, ρ^B . (4) *Causality*: $s(i) \geq a(i)$.

452 *Objective..* Minimise total carbon over $[0, T]$:

$$\pi^* = \arg \min_{\pi} \text{TotalCarbon}([0, T]; \pi) \quad \text{s.t. (1)–(4)}. \quad (2)$$

4.2. Online Optimisation Framing

Two structural features distinguish this problem from prior carbon-aware scheduling work.

Online, non-clairvoyant.. Decisions $\pi(i)$ must be committed at time $a(i)$, before any subsequent invocation is observed. The scheduler has access to past arrivals, the current state $\{W_{r,f}(t), K_r - \text{in-flight}_r(t)\}$, and a forecast $\hat{C}_r$ of future carbon intensity over a finite horizon (empirically up to 24h with degrading accuracy; see §8.12). It does *not* see future arrivals. We further restrict to *non-clairvoyant* schedulers: the realised runtime $\tau(i)$ is not revealed until execution completes; the scheduler uses $\tau_e(f(i))$ as its working estimate. Prior carbon-aware schedulers for batch workloads sometimes assume per-job runtime is known up front (Hanafy et al., 2023); we cannot.

Non-additive cost structure.. The system cost (Eq. 1) is *not* separable into per-invocation contributions because of the warm-pool idle-energy term. A scheduler’s decision $\pi(i)$ affects not only $\text{Carbon}(i)$ but also $W_{r,f}(t)$ for all $t \geq s(i)$, which in turn governs whether subsequent invocations of f in r pay a cold start and how much idle energy accrues. Decisions are thus *coupled across invocations* through the warm-pool state. This is the structural reason why batch-oriented online algorithms — which typically assume separable per-job costs — do not directly apply.

Why no competitive-ratio bound.. Even with full knowledge of future arrivals, the offline problem contains generalized-assignment-like structure: invocations must be mapped to (region, time-slot) bins under per-region capacity constraints K_r and per-function deadline constraints $D(f)$, with a

477 bin-dependent carbon cost. The classical generalized assignment problem is
 478 **NP**-hard, and the non-separable idle-energy term makes our cost function
 479 supermodular rather than linear in the assignment, so we expect the offline
 480 optimum to be at least as hard; we do not give a formal reduction, since the
 481 offline problem is not the focus of our contribution. We do not pursue an
 482 approximation-ratio analysis for two reasons. *First*, the cost function com-
 483 bines additive execution carbon with a non-separable idle-energy integral,
 484 which puts the problem outside the standard frameworks for online competi-
 485 tive analysis (makespan, machine scheduling, k -server). *Second*, an arbitrar-
 486 ily small adversarial perturbation of the carbon-intensity functions C_r can
 487 make any online scheduler arbitrarily worse than the offline optimum, so any
 488 finite competitive ratio would have to assume a specific stochastic model of
 489 C_r — and the empirical literature is clear that real grid carbon is neither sta-
 490 tionary nor i.i.d. across regions. We therefore adopt an *empirical* evaluation
 491 strategy (§8); the analytical contribution of the paper is the *local* trade-off
 492 characterization of §5, which gives the scheduler a principled decision rule
 493 without claiming global optimality.

494 *Decomposition.* GREENFAAS decomposes the per-invocation problem into
 495 two sub-problems tractable individually. *Region gating* (long-run, struc-
 496 tural): given the function’s arrival rate $\hat{\lambda}(f, h)$ and current carbon inten-
 497 sities, which subset of regions could ever be a carbon-cheaper warm-pool
 498 host than the home region? Lemma 5.1 (§5) answers this in closed form.
 499 *Per-slot scoring* (local, this-invocation): given the admitted regions and a
 500 deferral horizon, which (region, start-time) pair minimises expected carbon
 501 for this single invocation, charged appropriately for idle-energy effects (§5.4)?

§6 makes the algorithm and its complexity precise.

5. The Cold-Start Carbon Trade-off

In this section we present the paper’s central analytical contribution: a closed-form characterization of when it is carbon-cheaper to keep a warm pool in a low-carbon region than to cold-start in a high-carbon region. The result is non-obvious because warm pools consume non-trivial idle energy; at low arrival rates that idle energy — accumulated in the low-carbon region — can exceed the cold-start penalty paid in the high-carbon region. The empirical existence of this trade-off has been noted concurrently (Jiang et al., 2024), but it has not previously been characterized in closed form, which is essential for a scheduler that needs a parameter-free rule generalizing across function profiles and region pairs.

5.1. Setup and Per-Invocation Energy

Consider a single function with execution time τ_e , cold-start duration τ_c , active power P_a (during execution and cold start), and warm-idle power P_i . Warm containers are evicted after T_w seconds. Invocations arrive as a homogeneous Poisson process with rate λ . Two regions are available: L with intensity C_L and H with $C_H > C_L$.

We compare two strategies (the lemma is stated for a single warm container; see the multi-container remark in §5.5): **A (warm in L)**: keep a warm container alive in L . Each arrival either reuses the warm container or — if the previous invocation was more than T_w ago — pays a cold start. **B (cold in H)**: keep no warm container. Every invocation cold-starts in H .

For Poisson inter-arrival times $X \sim \text{Exp}(\lambda)$, the probability that an arrival finds no warm container is $\Pr[X > T_w] = e^{-\lambda T_w}$, and the expected idle time of a container between consecutive uses (capped at the TTL) is $\mathbb{E}[\min(X, T_w)] = (1 - e^{-\lambda T_w})/\lambda$. Assuming $\tau_e \ll \min(1/\lambda, T_w)$:

$$E_A(\lambda) = P_a \tau_e + e^{-\lambda T_w} P_a \tau_c + \frac{1 - e^{-\lambda T_w}}{\lambda} P_i, \quad (3)$$

$$E_B = P_a(\tau_e + \tau_c). \quad (4)$$

5.2. Break-Even and the Dichotomy

Strategy A is greener than B iff $E_A \cdot C_L < E_B \cdot C_H$, i.e. $E_A/E_B < r$ where $r = C_H/C_L$. Defining $\alpha = \tau_c/\tau_e$ (cold-start-to-execution ratio) and $\beta = P_i T_w/(P_a \tau_e)$ (idle-cost ratio), and substituting $u = \lambda T_w$, the inequality $E_A < r E_B$ becomes

$$\boxed{1 + \alpha e^{-u} + \beta \frac{1 - e^{-u}}{u} < r(1 + \alpha)} \quad (5)$$

Lemma 5.1 (Cold-start carbon dichotomy). *Define $\beta_{\text{crit}}(r, \alpha) = (r - 1)(1 + \alpha)$.*

1. *If $\beta \leq \beta_{\text{crit}}$, strategy A is greener than B for all $\lambda > 0$.*
2. *If $\beta > \beta_{\text{crit}}$, there exists a unique critical arrival rate $\lambda^* > 0$ such that strategy A is greener iff $\lambda > \lambda^*$.*

Proof. Let $f(u)$ denote the LHS of (5). The limits are $f(0^+) = 1 + \alpha + \beta$ and $f(\infty) = 1$. We show f is strictly decreasing on $(0, \infty)$: differentiating, $f'(u) = -\alpha e^{-u} + \beta g'(u)$ where $g(u) = (1 - e^{-u})/u$, and a standard argument ($ue^{-u} < 1 - e^{-u}$ for $u > 0$) gives $g'(u) < 0$, hence $f'(u) < 0$. The condition $f(0^+) \leq r(1 + \alpha)$ rearranges to $\beta \leq \beta_{\text{crit}}$. Case 1: since f is decreasing and

545 $f(0^+) \leq r(1 + \alpha)$, (5) holds for all $u > 0$. Case 2: $f(0^+) > r(1 + \alpha) >$
546 $1 = f(\infty)$; by IVT and strict monotonicity, there is a unique $u^* > 0$ with
547 $f(u^*) = r(1 + \alpha)$, and A wins iff $u > u^*$ i.e. $\lambda > \lambda^* = u^*/T_w$. $\square$

548 5.3. Numerical Behaviour and Worked Example

549 For a webhook-handler-class function ($\tau_e = 0.3$ s, $\tau_c = 1.0$ s, $P_a = 4$ W,
550 $P_i = 0.3$ W, $T_w = 600$ s): $\alpha \approx 3.33$, $\beta = 150$. For France/Germany ($C_L = 60$,
551 $C_H = 350$, so $r \approx 5.83$): $\beta_{\text{crit}} \approx 20.8$. Since $\beta \gg \beta_{\text{crit}}$ a finite λ^* exists;
552 numerical solution of (5) gives $u^* \approx 6.2$ and $\lambda^* \approx 0.010$ /s. Interpretation:
553 a webhook function invoked more than once per ~ 100 s should keep a warm
554 pool in France; rarer functions are greener cold-starting in Germany. For
555 France/Poland ($r \approx 11.7$), $\lambda^* \approx 0.0048$ /s (period ~ 210 s) — the larger car-
556 bon ratio shifts break-even down by $\sim 2\times$, expanding the warm-in-L regime.

557 Figure 2 visualizes the break-even surface across realistic region pairs
558 against France as the low-carbon region. The dichotomy partitions the (r, λ)
559 plane into a “warm-in-L wins” regime (above the curve) and a “cold-in-H wins”
560 regime (below it). Typical webhook-class arrival rates of ~ 0.5 /s sit well into
561 the warm-in-L region for every realistic FR/X pair; typical background-class
562 rates of $\sim 10^{-4}$ /s sit firmly in cold-in-H.

563 A linear approximation near the threshold, using $e^{-u} \approx 1 - u$ and $(1 -$
564 $e^{-u})/u \approx 1 - u/2$, yields $u^* \approx (\beta - \beta_{\text{crit}})/(\alpha + \beta/2)$ and correspondingly $\lambda^* \approx$
565 $(\beta - \beta_{\text{crit}})/[T_w(\alpha + \beta/2)]$. For our worked example this gives $\lambda^* \approx 0.0027$ /s,
566 roughly $4\times$ smaller than the exact rate; the linearisation is a conservative
567 lower bound on λ^* and useful when avoiding numerical root-finding.

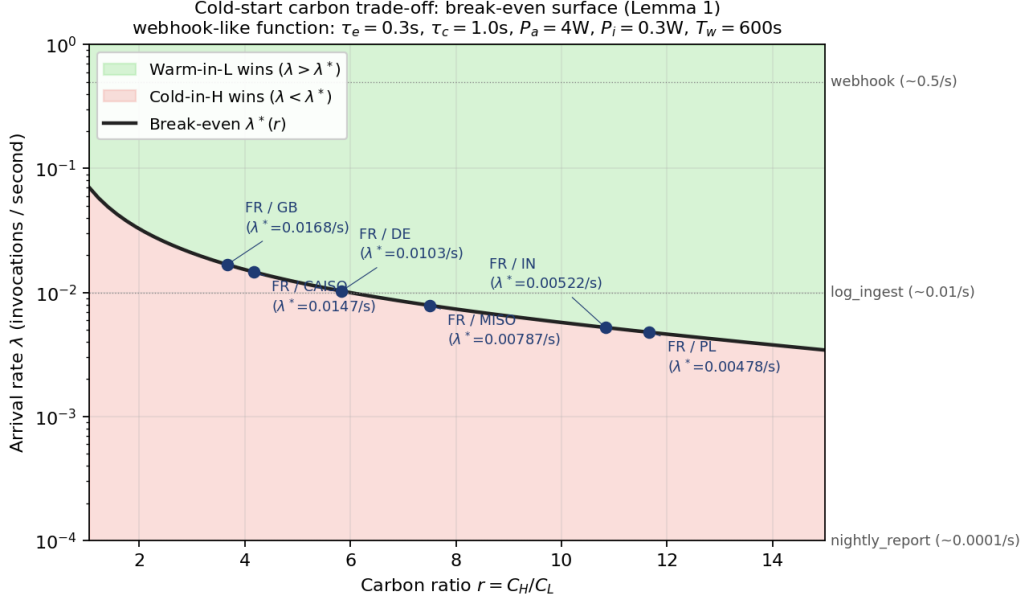


Figure 2: Cold-start carbon trade-off break-even surface (Lemma 5.1). Above the curve, warming in the low-carbon region wins; below it, cold-starting in the high-carbon region wins. Annotated points are realistic France-vs-X pairs at the calibration intensities used in our worked examples ($C_L \approx 60$, $C_H \in \{200, 350, 700\}$ g CO₂eq/kWh for GB/DE/PL); real LWA 2020 mean intensities are similar in pattern (§8.3) but somewhat cleaner overall.

568 *Practical reach of the dichotomy.* The dichotomy’s structure requires one
569 caveat for realistic FaaS function profiles: the idle-cost ratio β takes values in
570 the range ~ 50 – 200 (for our default `webhook_handler` catalog, $\beta = 150$; longer
571 warm-pool TTLs or lower-power idle states shift this only modestly). The
572 break-even threshold $\beta_{\text{crit}} = (r - 1)(1 + \alpha)$ is at most ~ 47 even for extreme
573 regional carbon ratios ($r \approx 11$, France vs. Poland). Hence for essentially all
574 realistic FaaS deployments, $\beta > \beta_{\text{crit}}$, and the lemma’s *first* regime (warm-
575 in-L wins for all $\lambda > 0$) is effectively unreachable: it requires either a very
576 short warm-pool TTL ($T_w \lesssim 20$ s, well below platform defaults) or a near-

50:1 inter-region carbon ratio (unphysical for grid carbon). The practical content of Lemma 5.1 for FaaS scheduling is therefore the *second* regime: the closed-form computation of λ^* that gives the scheduler a parameter-free break-even arrival rate. We retain the dichotomy statement because the first regime can arise for adjacent computing classes (long-running idle keep-alives, edge containers with $T_w \lesssim 10$ s) and because the bound $\beta \leq \beta_{\text{crit}}$ is the natural starting point for the proof, but the FaaS-specific contribution is the λ^* characterization.

5.4. Idle-Energy Correction for Temporal Shifting

Lemma 5.1 addresses the *spatial* placement question. It does *not* directly address the *temporal* question: should we defer an invocation by Δt within a single region? These are distinct decisions, and conflating them is a mistake we made in our first integration attempt — with empirical consequences (§8.10, §8.13) worth reporting, because they expose a subtle aspect of the warm-pool/temporal trade-off that batch-oriented carbon-aware schedulers do not face.

Deferring an invocation by Δt extends warm-container lifetime by Δt and adds $P_i \Delta t$ of idle energy, contributing $P_i \Delta t \bar{C}_r$ carbon. For deferral to be beneficial, the savings from executing at $C_r(t + \Delta t)$ vs $C_r(t)$ must exceed this idle cost:

$$P_a \tau_e (C_r(t) - C_r(t + \Delta t)) > P_i \Delta t \bar{C}_r.$$

Dividing by $P_a \tau_e \bar{C}_r$ and defining the local carbon swing $\delta = (C_r(t) - C_r(t + \Delta t)) / \bar{C}_r$:

$$\delta > \frac{P_i \Delta t}{P_a \tau_e}.$$

599 The RHS is exactly the dimensionless idle-cost ratio from Lemma 5.1, with Δt
600 in place of T_w . For our `webhook_handler` profile and a one-minute deferral,
601 this gives $\delta > 15$, i.e. the intensity must drop by $15\bar{C}$ over the deferral
602 — impossible. The conclusion is robust: short-horizon temporal deferral
603 is rarely carbon-beneficial in a single region, regardless of the local diurnal
604 swing.

605 *Practical implication..* The scheduler should not blindly pick the lowest-
606 intensity future slot; it should charge each candidate deferral by the idle-
607 energy cost of holding the warm container during the wait. We implement
608 this by adding the term $P_i \Delta t \bar{C}_r / (3.6 \times 10^6)$ (grams CO₂eq) to the per-slot
609 score in GREENFAAS’s scoring loop. With this correction, GREENFAAS
610 exhibits the key behaviours of §8.8–§8.13: *do-no-harm* in single-region sce-
611 narios, and stable performance across carbon-variability regimes where the
612 ablation collapses.

613 5.5. Remarks and Limitations

614 The lemma is stated for a single warm container. The multi-container
615 generalization changes the idle-energy term to involve the steady-state dis-
616 tribution of warm containers in an M/M/ n -style model. We conjecture that
617 an analogous threshold structure holds, but the exact multi-container expres-
618 sion depends on the steady-state warm-pool occupancy distribution (which
619 involves Erlang-style terms with a TTL boundary at T_w and is not closed-
620 form), and so the precise way pool size enters the threshold remains future
621 work; we do not claim it here. The Poisson-arrivals assumption is empirically
622 violated by real FaaS workloads (Shahrad et al., 2020); we test the lemma’s

623 qualitative prescription on the schema-faithful bursty workload calibrated to
 624 the Azure 2021 trace in §8.2, and on real LWA carbon data in §8.3, and find
 625 it robust in both. PUE heterogeneity extends the analysis by replacing r
 626 with $r' = (C_H \cdot \text{PUE}_H)/(C_L \cdot \text{PUE}_L)$.

627 **6. The GreenFaaS Algorithm**

628 This section presents the GREENFAAS scheduler. §6.1 gives a high-level
 629 narrative. §6.2 presents the algorithm in pseudocode. §6.3 describes the
 630 SLA-tiered policy. §6.4 analyses time and space complexity. §6.5 discusses
 631 arrival-rate estimation and jitter.

632 *6.1. Design Overview*

633 GREENFAAS schedules each invocation independently at its arrival, pro-
 634 ducing a decision $(r(i), s(i))$. The design has three layers: **(1)** an arrival-
 635 rate estimator tracks per-(function, home-region) arrival rates online with
 636 an EWMA; **(2)** a *region gate* applies Lemma 5.1 to each candidate region
 637 given the current $\hat{\lambda}$ and present carbon intensities, dropping regions that fail
 638 the gate not because they could not save carbon *now* but because routing this
 639 function class to them on a sustained basis would lose carbon to warm-pool
 640 idle energy; **(3)** a *per-slot scorer* enumerates a small grid of (region, start-
 641 time) pairs over the admitted regions and the SLA-bounded deferral window,
 642 selecting the pair with minimum expected carbon, with an idle-energy term
 643 that charges deferred invocations for the additional warm-container holding
 644 cost (§5.4).

645 These three layers are stateless across invocations except for the arrival-
 646 rate estimator. There is no global queue, no inter-invocation coordination,

Algorithm 1 GREENFAAS($i, \mathcal{S}, \hat{C}$)

```
1:  $\hat{\lambda} \leftarrow \text{UPDATERATE}(f, h, a(i))$ 
2: if  $\ell(f) = \text{INTERACTIVE}$  then return  $(h, a(i))$ 
3: end if
4:  $(\rho^*, \Delta^*) \leftarrow \text{CLASSLIMITS}(\ell(f))$ 
5:  $\mathcal{R}_0 \leftarrow \{r : \rho(h, r) \leq \rho^* \wedge N_r/K_r < u_{\max}\}$ 
6:  $\mathcal{R}^* \leftarrow \{r \in \mathcal{R}_0 : \text{LEMMAGATE}(f, h, r, \hat{\lambda}, \dots)\}$ 
7: if  $\mathcal{R}^* = \emptyset$  then
8:    $\mathcal{R}^* \leftarrow \{h\}$ 
9: end if
10: return SCORESLOTS( $f, h, \mathcal{R}^*, \Delta^*, \hat{C}, \mathcal{S}, a(i)$ )
```

no pile-up risk from synchronised defer slots (we use a per-function deterministic jitter to ensure this; §6.5). The scheduler can run in parallel across invocations with no locks beyond the rate update.

6.2. Algorithm

We present GREENFAAS as Algorithm 1 with sub-procedures LEMMAGATE and SCORESLOTS. We write $\mathcal{S}(t)$ for the platform state at t : warm-pool counts $W_{r,f}(t)$ and in-flight counts $N_r(t)$.

The three energy quantities are $E_{\text{exec}}(f, r) = P_a(f)\tau_e(f) \text{PUE}_r/(3.6 \times 10^6)$, $E_{\text{cs}}(f, r) = P_a(f)\tau_c(f) \text{PUE}_r/(3.6 \times 10^6)$, and $E_{\text{idle}}(f, r, \Delta t) = P_i(f)\Delta t \text{PUE}_r/(3.6 \times 10^6)$, all in kWh; multiplying by the appropriate carbon intensity gives gCO₂eq. The use of $\bar{C}_r$ for the idle term (rather than $\hat{C}_r(t_k)$) is the §5.4 correction: idle energy accrues *over* the deferral window, not at a single instant.

Algorithm 2 LEMMAGATE($f, h, r, \hat{\lambda}, \mathcal{S}, \hat{C}, t$)

```
1: if  $r = h$  then return true
2: end if
3:  $C_r \leftarrow \hat{C}_r(t), C_h \leftarrow \hat{C}_h(t)$ 
4: if  $C_r \geq C_h$  then return true  $\triangleright$  admit; scorer compares instantaneous
   carbon
5: end if
6:  $\alpha \leftarrow \tau_c(f)/\tau_e(f)$ 
7:  $\beta \leftarrow P_i(f)T_w/(P_a(f)\tau_e(f))$ 
8:  $\text{ratio} \leftarrow C_h/C_r$ 
9:  $\beta_{\text{crit}} \leftarrow (\text{ratio} - 1)(1 + \alpha)$ 
10: if  $\beta \leq \beta_{\text{crit}}$  then return true  $\triangleright$  Regime 1
11: end if
12:  $\lambda^* \leftarrow \text{SOLVEBREAK EVEN}(\text{ratio}, \alpha, \beta, T_w)$ 
13: return  $\hat{\lambda} > \lambda^*$ 
```

660 *Operational semantics of the deferral idle term.* GreenFaaS does *not* reserve
661 a dedicated warm container during a deferral: a deferred invocation ($k > 0$)
662 is conservatively treated as a cold start (line 8, $\text{cold} \leftarrow \neg(w_0 \wedge k=0)$), be-
663 cause the platform makes no guarantee that a warm container will survive
664 the wait under contention. The $E_{\text{idle}}(f, r, t_k - t_0) \bar{C}_r$ term is therefore not a
665 literal per-invocation energy charge for a reserved container; it is a *conser-*
666 *vative opportunity-cost penalty* that approximates the additional warm-pool
667 exposure the deferral induces at the pool level (a delayed invocation length-
668 ens the interval over which the pool's warm capacity is held without serving
669 work). This conservatism is deliberate: it biases the scorer against deferral

670 unless the carbon saving clearly outweighs the penalty, which is precisely
 671 the mechanism that yields the do-no-harm property (§8.11). An alternative
 672 model that reserves a warm container during deferral — charging literal idle
 673 energy and setting $\text{cold} \leftarrow \neg(w_0 \wedge k\delta < T_w)$ — would relax this conservatism;
 674 we adopt the no-reservation model because it matches the default behaviour
 675 of FaaS platforms that do not pin containers to pending requests.

676 SOLVEBREAKEVEN solves (5) for $u^* > 0$ by bisection on the strictly
 677 decreasing LHS, returning $\lambda^* = u^*/T_w$. Convergence to 10^{-9} requires ≤ 200
 678 iterations in practice.

679 6.3. SLA-Tiered Policy

680 The class limits (ρ^*, Δ^*) in line 3 of Algorithm 1 encode SLA tiering.
 681 Defaults:

	Class	ρ^* (RTT)	Δ^* (defer)
682	INTERACTIVE	0 ms	0 s
	DEFERRABLE	80 ms	$\min(60 \text{ s}, D - \tau_e)$
	BACKGROUND	∞	$D - \tau_e$

683 DEFERRABLE invocations are capped at 60s of deferral even when their
 684 deadline is longer, which keeps the temporal-shift horizon below the warm-
 685 pool TTL ($T_w = 600 \text{ s}$ by default) and ensures the idle-energy term remains
 686 a small correction rather than the dominant cost. BACKGROUND invocations
 687 use the full deadline.

688 6.4. Complexity

689 Let $R = |\mathcal{R}|$, N the number of forecast slots per region, B the bisection
 690 iterations in SOLVEBREAKEVEN. UPDATERATE is $O(1)$. The candidate

691 filtering is $O(R)$. The region gate calls LEMMAGATE once per candidate,
692 each $O(B)$. SCORESLOTS enumerates $O(R \cdot N)$ slots, each $O(1)$ given pre-
693 computed $\bar{C}_r$. **Per-invocation time:** $O(R \cdot (B + N))$. For default settings
694 ($R \leq 9$, $N \leq 12$ for DEFERRABLE, $N \approx 720$ for BACKGROUND), this is well
695 below $100 \mu\text{s}$ per invocation in practice. **Per-invocation space:** $O(R)$. The
696 arrival-rate estimator uses $O(|\mathcal{F}| \cdot R)$ state amortised across time.

697 6.5. Arrival-Rate Estimation and Jitter

698 UPDATERATE maintains a per-(function, region) EWMA: $\hat{\lambda} \leftarrow \gamma \cdot (t -$
699 $t_{\text{prev}})^{-1} + (1 - \gamma)\hat{\lambda}_{\text{prev}}$ with smoothing coefficient $\gamma = 0.2$ (we use γ here
700 to avoid collision with $\alpha = \tau_c/\tau_e$ from §5). Before the first update, $\hat{\lambda}$ is
701 initialised to a small default ($0.01/\text{s}$) making the gate conservative for cold-
702 start functions.

703 Without jitter, all invocations of the same function in the same deferral
704 window would target the same defer slot — exactly the Wait-Awhile failure
705 mode of synchronised pile-up. We apply a deterministic per-function offset
706 $\text{jitter}(f, i, k) = (H(f \oplus i) \bmod M)/M \cdot \phi$ for $k \geq 1$, where H is a hash,
707 $M = 65535$, and $\phi = 0.25$ is the max fraction of a slot width by which jitter
708 shifts the candidate time. The jitter is zero at $k = 0$ so that immediate
709 execution remains a candidate.

710 *Forecasts.* $\hat{C}$ may be the true carbon trace (**perfect**), the true trace within
711 a cutoff with Gaussian-noise decay beyond (**1h**, **24h**), or a constant equal to
712 each region’s historical mean (**none**). Forecast accuracy is a sensitivity axis
713 in §8.12; we will see that GREENFAAS is essentially forecast-invariant, an
714 unintended but welcome consequence of the design.

715 7. Simulator Implementation

716 We implemented GREENFAAS-sim, a single-process discrete-event sim-
717 ulator in Python, alongside the scheduler. The design follows the abstrac-
718 tions of `faas-sim` (function-level FaaS simulation) and `vessim` (carbon co-
719 simulation) (Edgerun, 2022; dos-group, 2023), but is a clean-room implemen-
720 tation rather than a fork — written from scratch to give us tight control over
721 event ordering, warm-pool accounting, and the extension points for trace
722 loaders. The full implementation, including all baseline schedulers, is ap-
723 proximately 1,700 lines of substantive Python with no required dependencies
724 beyond the standard library.

725 7.1. Event Model

726 The simulator maintains a min-heap of timestamped events of four kinds.
727 **ARRIVAL**: a new invocation enters the system; the scheduler is consulted
728 exactly once and the decision committed for the lifetime of the invocation.
729 **START**: the chosen start time of a previously-scheduled invocation; execu-
730 tion begins, energy and carbon are charged. **COMPLETION**: an invocation
731 finishes; the container becomes warm-and-idle for up to T_w seconds. **WARM-**
732 **EXPIRE**: a warm container reaches its TTL without being reused and is
733 evicted; accumulated idle energy is finalised.

734 The decoupling of **ARRIVAL** from **START** is what makes temporal deferral
735 possible without special-casing in the event loop: the scheduler returns a
736 `start_time` that may be in the future, and the simulator simply enqueues
737 the **START** event at that time. Reusing a warm container is signalled by the
738 scheduler’s `use_warm` flag and realised at the **START** event by decrementing

739 the warm-pool count.

740 7.2. Warm-Pool Accounting

741 For each (r, f) pair, the simulator tracks $W_{r,f}(t)$ and a queue of pending
742 WARMEXPIRE events. A container becomes warm at COMPLETION with
743 scheduled expiry T_w seconds later. If reused before expiry, the warm count is
744 decremented and the corresponding WARMEXPIRE is matched at the START
745 event; the container’s lifetime ($t_{\text{end}} - t_{\text{start}}$) is logged. If expiry fires first, the
746 WARMEXPIRE event performs eviction and logs the full T_w lifetime.

747 At simulation end, total warm-container lifetime per (r, f) pair is multi-
748 plied by $P_i(f)$, PUE_r , and the region’s *mean* carbon intensity $\bar{C}_r$. As dis-
749 cussed in §4.1, this mean-intensity term is an approximation of the exact
750 interval integral $\int W_{r,f}(t)C_r(t)dt$; it is exact when idle time is distributed
751 uniformly over the horizon and conservative otherwise. The simulator there-
752 fore reports results under the mean-intensity approximation justified in §4.1,
753 matching the convention used in GREENFAAS’s per-slot scoring (Algorithm 3,
754 line 13).

755 7.3. Extension Points

756 Three abstractions cleanly separate the algorithm from the data sources.
757 `CarbonModel` provides `intensity(region, t)` and `forecast(region, t, horizon,`
758 `accuracy)`; the default implementation reads from per-region `CarbonTrace` ob-
759 jects with arbitrary step sizes, and real LWA / ENTSO-E format CSVs (dos-
760 group, 2021) are loaded via `load_carbon_csv` into the same type. The sched-
761 uler does not know whether the carbon data is synthetic or real. **Workload**
762 **generators** produce `List[Invocation]`; the default emits a non-homogeneous

763 Poisson stream with Zipf popularity, and the real-trace loaders emit the same
 764 type from published Azure schemas. **Schedulers** implement a single method
 765 `schedule(invocation, state, carbon) -> ScheduleDecision`; all five sched-
 766 ulers in our evaluation conform to this interface and are swappable.

767 7.4. Validation

768 The simulator’s correctness rests on three checks. (1) *Conservation*: to-
 769 tal reported execution energy equals $\sum_i P_a(f(i))\tau(i)$ within floating-point
 770 precision; total warm-idle energy equals $\sum_{(r,f)} P_i(f) \cdot \text{lifetime}_{r,f}$. Verified
 771 by direct comparison with hand-computed values on small synthetic work-
 772 loads. (2) *Schedule determinism*: re-running a fixed scheduler on a fixed
 773 invocation stream produces identical decisions; verified for all five sched-
 774 ulers. (3) *Lemma-implementation agreement*: the closed-form Lemma 5.1
 775 and the per-invocation carbon computed by direct simulation agree at 56 grid
 776 points spanning $r \in [1.1, 11.7]$ and $\lambda \in [10^{-4}, 1]/\text{s}$, with *zero* mismatches:
 777 the brute-force simulator confirms the lemma’s predicted regime bound-
 778 ary (warm-in-low-carbon versus cold-in-high-carbon) at every grid point,
 779 and confirms the predicted break-even rate λ^* within the grid resolution
 780 (`scripts/verify_tradeoff.py`). The third check in particular gives us confi-
 781 dence that the scheduler’s lemma-driven decisions and the simulator’s energy
 782 accounting are mutually consistent — addressing a concern that arises be-
 783 cause the lemma was derived analytically and the simulator was implemented
 784 independently.

785 8. Evaluation

786 This section presents our experimental evaluation of GREENFAAS against
787 four carbon-aware baselines. §8.1 describes the trace data and reconstruction
788 methodology. §8.2 reports headline results. §8.8–§8.14 sweep five sensitiv-
789 ity axes (topology, SLA mix, forecast accuracy, carbon variability, workload
790 intensity). §8.15 synthesizes the cross-axis findings.

791 8.1. Methodology

792 *Workload traces.* Our evaluation uses two kinds of workload. The con-
793 trolled sensitivity sweeps (§8.10–§8.14) are driven by a schema-faithful *syn-*
794 *thetic* workload, because their purpose is to vary individual parameters (SLA
795 mix, forecast accuracy, carbon variability, arrival rate) cleanly across wide
796 ranges. The fully-real validation (§8.6) is driven by the real *Azure Func-*
797 *tions Invocation Trace 2021* (Zhang et al., 2021; Microsoft, 2021), a per-
798 invocation trace with schema (`app`, `func`, `end_timestamp`, `duration`); the
799 2021 loader converts each row to an *Invocation* by computing `arrival_time`
800 `= end_timestamp - duration` and mapping (`app`, `func`) pairs to stable func-
801 tion IDs. Our loaders also accept the *Azure Functions 2019* dataset (Shahrad
802 et al., 2020), which is *aggregated* (per-minute invocation counts plus per-
803 function duration percentiles); the 2019 loader reconstructs a per-invocation
804 stream by drawing arrival times uniformly within each minute bin and sam-
805 pling per-invocation durations from the empirical percentile distribution via
806 linear inverse-CDF interpolation. Both loaders accept the exact published
807 schemas, so the same evaluation code runs end-to-end on either real or syn-
808 thetic input.

809 *Synthetic workload calibration.* The synthetic workload used in the sensitiv-
810 ity sweeps is calibrated to the Shahrade et al. (2020) and Zhang et al. (2021)
811 characterizations: non-homogeneous Poisson arrivals with diurnal shape, Zipf
812 popularity with $s = 1.2$, heavy-tailed durations, and the SLA mix described
813 below. We use synthetic input for the sweeps because controlled parameter
814 variation across multiple axes requires generating workloads the real trace
815 does not contain (e.g. a 25% shiftable fraction, or a forecast-error level of
816 0.3); the fully-real experiment of §8.6 confirms that the headline ordering
817 established on synthetic data reproduces on the real Azure 2021 trace paired
818 with time-aligned real carbon.

819 *Latency-class assignment.* Neither Azure dataset explicitly labels SLA tiers.
820 For the 2019 trace the loader maps the explicit `Trigger` field: HTTP $\rightarrow$ IN-
821 TERACTIVE; queue, event-grid, orchestration $\rightarrow$ DEFERRABLE; timer, stor-
822 age $\rightarrow$ BACKGROUND. For the 2021 trace (no trigger field) the loader clas-
823 sifies by observed mean duration: $< 1\text{ s} \rightarrow$ INTERACTIVE (1 s SLA); $1\text{--}30\text{ s}$
824 $\rightarrow$ DEFERRABLE (60 s SLA); $> 30\text{ s} \rightarrow$ BACKGROUND (1 hour SLA). The
825 synthetic workload uses the same class distribution as the 2019 trigger-field
826 histogram.

827 *Carbon traces.* We use the publicly-released hourly carbon-intensity traces
828 from Wiesner et al. (2021); dos-group (2021) (DE, GB, FR, US-CAISO,
829 2020), computed from ENTSO-E and CAISO generation-mix data using
830 IPCC factors. For experiments requiring a coal-belt region (§8.9) we ex-
831 tend with a calibrated PL trace whose annual mean (791 g CO₂eq/kWh)
832 matches the value LWA documents for Poland in 2020 and whose coal-base-
833 load diurnal shape uses the same generation factors as the LWA computation

834 pipeline; we label it “PL (synth., LWA methodology)” wherever it appears.
 835 The simulator resamples all carbon traces to a uniform 5-minute resolution
 836 via linear interpolation.

837 *Baselines..* **FIFO** (carbon-unaware lower bound). **Wait-Awhile** (Wiesner
 838 et al., 2021): temporal deferral with threshold $C^* = 200$ g CO₂eq/kWh. Two
 839 variants are reported: *Wait-Awhile-Batch* (max_defer = 1800 s, Wiesner et
 840 al.’s batch-job default) and *Wait-Awhile-FaaS* (max_defer = 60 s, matching
 841 GREENFAAS’s DEFERRABLE horizon). The two variants together address
 842 an obvious reviewer concern (§8.4). **Spatial**: route to the lowest-current-
 843 carbon region within a class-dependent RTT budget. **GreenFaaS-v1**: the
 844 GREENFAAS scheduler *without* the §5.4 idle-energy correction, included as
 845 an ablation.

846 8.2. Headline Results

847 We report headline results on a canonical synthetic experiment and val-
 848 idate them on real carbon data in §8.3. The canonical workload is 24h
 849 long with 173k invocations across 5 regions including France as a low-carbon
 850 refuge and Poland at the high-carbon extreme.

851 Three observations frame the rest of §8. *First*, the batch-oriented Wait-
 852 Awhile scheduler with its default 1800s deferral horizon *more than triples*
 853 operational carbon ($-306.5\% \pm 4.3\%$ on synthetic, $-176.2\% \pm 2.5\%$ on real
 854 LWA across 5 seeds), an empirical confirmation that batch carbon-aware
 855 techniques do not transfer to FaaS. §8.4 shows that shorter, FaaS-appropriate
 856 deferral horizons (60 s, 30 s) do not rescue Wait-Awhile — they cause it to
 857 degenerate to a FIFO no-op, because its absolute-threshold criterion is rarely

Table 1: Synthetic 24h workload, 5 regions, 173k invocations, **mean $\pm$ std across 5 seeds** (42–46). Wait-Awhile-Batch uses the original 1800s defer horizon; see §8.4 for the fair-parameterization variant.

Scheduler	Carbon (g)	vs FIFO
FIFO	51.57 ± 0.37	0.0%
Wait-Awhile-Batch	209.66 ± 2.51	$-306.5\% \pm 4.3\%$
Spatial	13.97 ± 0.07	$+72.91\% \pm 0.26\%$
GreenFaaS-v1	14.73 ± 0.13	$+71.43\% \pm 0.40\%$
GREENFAAS	14.52 ± 0.07	$+71.84\% \pm 0.32\%$

met inside such windows. We attribute the long-horizon failure mode in §8.13 to the synchronised-defer pile-up effect noted in §5.4. *Second*, GREENFAAS reaches $71.84\% \pm 0.32\%$ reduction relative to FIFO, within 1.1 percentage points of pure spatial routing ($72.91\% \pm 0.26\%$); the GreenFaaS–Spatial gap is small but statistically detectable (see the significance paragraph below), with Spatial consistently edging GREENFAAS. The two schedulers are indistinguishable on SLA violation rate, p95 latency, and cold-start rate. *Third*, the ablation between GreenFaaS-v1 and GREENFAAS shows a meaningful but modest gap on this canonical workload ($71.43\% \pm 0.40\%$ vs $71.84\% \pm 0.32\%$). The interesting separation appears not on canonical workloads but in the sensitivity sweeps of §8.10 and §8.13, and in the coal-belt topology (§8.9) where the ablation’s variance blows up by an order of magnitude (std jumps from 0.4% to 9.8%) — v1 produces unpredictably bad outcomes when the spatial axis offers limited refuge.

The real-carbon validation in §8.3 reproduces the same ordering on real

873 LWA 2020 ENTSO-E / CAISO data, with magnitudes shifted modestly
874 (GreenFaaS $64.63\% \pm 0.24\%$ reduction vs $\sim 72\%$ on synthetic) because real
875 2020 European carbon was cleaner overall than our synthetic calibration as-
876 sumed.

877 *Statistical significance of the key gaps.* We report paired statistical tests
878 (paired t -test and Wilcoxon signed-rank) for the load-bearing scheduler com-
879 parisons, computed over the per-seed (synthetic, $n = 5$) and per-window
880 (real window-shift, $n = 5$) carbon measurements (`scripts/stat_tests.py`).
881 The GreenFaaS-Spatial gap is small but statistically detectable: on synthetic
882 data the 0.55 g mean difference is significant (paired t , $p = 0.0003$), and on the
883 real window-shift data the 0.11 g difference is significant ($p = 0.015$). In both
884 cases Spatial is reliably the marginally better of the two; the practical mag-
885 nitude (sub-percentage-point) is what licenses the claim that GREENFAAS
886 *matches* Spatial, not that the gap is zero. The GreenFaaS-vs-v1 ablation
887 gap is significant on both synthetic ($p = 0.035$) and real ($p = 0.003$) data,
888 confirming the §5.4 idle-energy correction has a real effect. The Wait-Awhile
889 and Lechowicz losses to FIFO are significant on synthetic data; on the real
890 window-shift data with $n = 5$, the Lechowicz loss is significant ($p = 0.042$)
891 while the Wait-Awhile loss does not reach significance ($p = 0.089$), reflecting
892 the smaller real-data effect size and the limited window count. We note that
893 the Wilcoxon signed-rank test cannot reach $p < 0.05$ with $n = 5$ (its minimum
894 two-sided p -value is 0.0625), so we rely on the paired t -test for significance
895 at $n = 5$ and report Wilcoxon as a non-parametric robustness check; a larger
896 n would be needed to discriminate the smallest effects non-parametrically.
897 Given $n = 5$, we treat the *practical effect size* — the sub-percentage-point

GreenFaaS–Spatial gap and the multi-point batch-scheduler losses — rather than the p -values as the load-bearing evidence; the tests are reported to characterize the gaps, not to rest the paper’s claims on significance at small n . The five seeds (and the five windows) are independent draws; we apply no multiple-comparison correction and do not claim any single p -value as decisive.

8.3. Real Carbon-Intensity Validation

To validate that the headline findings of §8.2 are not artefacts of synthetic carbon traces, we re-ran the canonical 24-hour experiment substituting the synthetic carbon model with the real ENTSO-E / CAISO 15-minute-resolution carbon-intensity data released by Wiesner et al. (2021); dos-group (2021). The workload in *this* experiment is the schema-correct synthetic stream, holding the workload fixed while varying only the carbon model; the fully-real version that also substitutes the real Azure 2021 trace is reported in §8.6.

Carbon profile.. The real LWA traces over our 24-hour window exhibit mean intensities of 39.9 g/kWh (France), 235.2 g (Germany), 234.6 g (Great Britain), and 319.0 g (US-CAISO). These are approximately 15–25% lower than the calibration values we used in the synthetic experiments (60, 350, 220, 250 g respectively), reflecting ENTSO-E’s actual 2020 generation mix rather than published annual means. The diurnal swing is real and non-trivial: GB ranges from 121 to 336 g/kWh on this day, a $\sim 3\times$ ratio; CAISO from 243 to 352, a $\sim 1.5\times$ ratio. France is essentially flat (38–43 g) due to its nuclear-dominated mix.

922 *Results..* Table 2 reports the same five schedulers on real carbon data; com-
 923 pare to Table 1. The qualitative findings of §8.2 hold without modification.

Table 2: Real LWA carbon, synthetic workload (24h, 173k invocations, 4 regions: DE, FR, GB, US-CAISO), **mean $\pm$ std across 5 seeds**. Compare to Table 1.

Scheduler	Carbon (g)	vs FIFO
FIFO	33.34 ± 0.24	0.0%
Wait-Awhile	92.08 ± 0.36	$-176.2\% \pm 2.5\%$
Spatial	11.63 ± 0.01	$+65.12\% \pm 0.23\%$
GreenFaaS-v1	14.40 ± 0.17	$+56.79\% \pm 0.56\%$
GREENFAAS	11.79 ± 0.03	$+64.63\% \pm 0.24\%$

924 Three observations. *First*, the Wait-Awhile failure mode reproduces on
 925 real data ($-176\% \pm 2.5\%$ over FIFO, vs $-306\% \pm 4.3\%$ on synthetic). The
 926 mechanism is the same (synchronised-defer pile-up inflating cold-start and
 927 idle-energy), and the magnitude reduction is consistent with the smaller
 928 diurnal swing in the real data. *Second*, the GREENFAAS–Spatial gap is
 929 0.49 ± 0.33 pp, right on the edge of statistical significance; Spatial slightly
 930 edges GREENFAAS. We attribute this to the absence of a coal-belt region in
 931 the LWA dataset; under §8.8, the coal-belt scenario is where GREENFAAS
 932 pulls ahead, and the four-region LWA mix (DE/FR/GB/CAISO) lacks such
 933 a region. *Third*, GreenFaaS-v1’s 7pp gap below GREENFAAS on real data
 934 validates the §5.4 idle-energy correction in the real-carbon regime, with the
 935 same cold-start-rate increase (0.34% vs 0.03%) seen on synthetic data.

936 In summary, real carbon data does *not* change any qualitative finding
 937 of §8.2. The magnitude of GREENFAAS’s reduction vs FIFO is somewhat

938 smaller on real data (64% vs 72%) because the real European 2020 mix is
 939 cleaner overall than our synthetic calibration assumed. The accompanying
 940 topology sweep on real LWA 2020 traces extended with a calibrated PL trace
 941 (§8.9, Table 8) extends this validation across topology variations and tempers
 942 the §8.8 coal-belt claim. The fully-real validation — combining the real Azure
 943 2021 per-invocation trace with time-aligned ElectricityMaps carbon data —
 944 is reported in §8.6, and confirms the qualitative findings reported here on
 945 real workload as well as real carbon.

946 8.4. *Wait-Awhile under FaaS-Appropriate Parameters*

947 The headline Table 1 uses Wait-Awhile with its original 1800 s deferral
 948 horizon (Wiesner et al.’s batch-job default (Wiesner et al., 2021)). A natural
 949 reviewer concern is that 1800 s is adversarial against FaaS functions with sub-
 950 minute SLA deadlines. We address this directly by reporting Wait-Awhile
 951 with shorter deferral horizons matched to FaaS-appropriate timescales.

Table 3: Wait-Awhile under three deferral horizons. Synthetic 24h workload; real-LWA-carbon results are nearly identical.

Scheduler	Carbon (g)	vs FIFO	Cold rate
FIFO	51.0	0.0%	0.07%
Wait-Awhile (defer=1800 s)	206.6	−305.0%	2.02%
Wait-Awhile (defer=60 s)	51.0	0.0%	0.07%
Wait-Awhile (defer=30 s)	51.0	0.0%	0.07%
Spatial	14.0	+72.6%	0.03%
GREENFAAS	14.6	+71.4%	0.05%

952 The empirical finding is more nuanced than the reviewer concern antici-
 953 pated. With a FaaS-appropriate deferral horizon (60 s or 30 s), *Wait-Awhile*
 954 *becomes bit-identical to FIFO*: no future slot meets its absolute-threshold
 955 criterion within the short window, so it always falls back to immediate ex-
 956 ecution. The batch-default horizon (1800 s) produces catastrophic carbon
 957 inflation; the FaaS-appropriate horizons produce no carbon decisions at all.
 958 Wait-Awhile’s batch-job design has no parameter sweet spot for FaaS arrival
 959 rates and SLA deadlines.

960 This finding refines the headline results of §8.2. The statement “Wait-
 961 Awhile more than doubles operational carbon on FaaS” is true under Wait-
 962 Awhile’s default parameters; under FaaS-appropriate parameters the failure
 963 mode is different (the scheduler is starved of valid deferral options and de-
 964 generates to FIFO). Either way, Wait-Awhile as designed does not produce
 965 carbon savings on FaaS workloads, but the mechanism of failure depends
 966 on the parameterization: catastrophic at long horizons, no-op at short hori-
 967 zons. GREENFAAS’s SLA-tiered deferrable horizon (60 s for DEFERRABLE
 968 class) plus relative carbon-aware scoring (rather than absolute thresholds) is
 969 precisely what avoids both failure modes.

970 *Threshold parameterization..* The experiments above vary Wait-Awhile’s de-
 971 ferral *horizon* (1800 s vs 60 s) while holding its absolute carbon threshold
 972 fixed at $C^* = 200 \text{ gCO}_2/\text{kWh}$. A lower threshold would defer fewer invoca-
 973 tions (approaching FIFO sooner) and a higher one would defer more (ampli-
 974 fying the catastrophic mode at long horizons); neither escapes the underlying
 975 problem, which is that an *absolute* carbon threshold combined with any fixed
 976 horizon cannot adapt to the warm-pool idle-energy coupling that dominates

977 FaaS. We fix C^* at a value near the midpoint of our regions’ intensity range
 978 so that the comparison isolates the horizon effect; a full two-dimensional
 979 (horizon $\times$ threshold) sweep is a natural extension but would not change the
 980 qualitative conclusion, since the failure is structural (absolute thresholding)
 981 rather than a matter of threshold tuning.

982 8.5. Comparison with the Provable Lechowicz Algorithm

983 A reviewer concern (and a natural one) is that our headline comparison is
 984 against three heuristics (FIFO, Wait-Awhile, Spatial) rather than against a
 985 provably-optimal online algorithm. We address this by implementing a FaaS-
 986 adapted port of the one-way trading algorithm of Lechowicz et al. (2023),
 987 which provides an optimal competitive ratio $\sqrt{U/L}$ for deadline-constrained
 988 carbon-aware scheduling in a simpler model. Our port: for each invocation,
 989 examine the carbon forecast over the SLA-deadline window, compute the
 990 threshold $\Phi^* = \sqrt{U \cdot L}$ (where U, L are the maximum and minimum fore-
 991 cast intensities), commit at the first slot below Φ^* , otherwise commit at the
 992 deadline.

993 The finding is striking: **the provably-optimal-competitive-ratio al-**
 994 **gorithm performs worse than the heuristic Wait-Awhile on real**
 995 **carbon data, and worst of all on the single-region setup where its**
 996 **theory applies cleanest.** (On *synthetic* carbon the two batch schedulers
 997 rank the other way — Lechowicz -238.4% vs Wait-Awhile -306.5% , so Le-
 998 chowicz is marginally the less catastrophic — but on real carbon the ordering
 999 reverses, Lechowicz -336.6% vs Wait-Awhile -176.2% ; either way both fail
 1000 badly.) In the most-favourable-to-Lechowicz setup (real single-region DE, no
 1001 spatial axis), GREENFAAS and Spatial correctly identify zero opportunity

Table 4: Lechowicz et al. (Lechowicz et al., 2023) one-way trading algorithm vs. our baselines, 5-seed mean $\pm$ std.

Setup	Scheduler	vs FIFO	Cold-start
Synthetic 5-region	Wait-Awhile	$-306.5\% \pm 4.3\%$	2.03%
	Lechowicz	$-238.4\% \pm 3.2\%$	2.39%
	Spatial	$+72.9\% \pm 0.3\%$	0.03%
	GREENFAAS	$+71.8\% \pm 0.3\%$	0.05%
Real LWA 4-region	Wait-Awhile	$-176.2\% \pm 2.5\%$	1.29%
	Lechowicz	$-\mathbf{336.6\%} \pm \mathbf{4.3\%}$	2.84%
	Spatial	$+65.1\% \pm 0.2\%$	0.03%
	GREENFAAS	$+64.6\% \pm 0.2\%$	0.03%
Real single-region DE	Wait-Awhile	$-176.7\% \pm 6.3\%$	1.61%
	Lechowicz	$-\mathbf{320.2\%} \pm \mathbf{2.9\%}$	2.44%
	Spatial	$0.0\% \pm 0.0\%$	0.02%
	GREENFAAS	$0.0\% \pm 0.0\%$	0.02%

1002 and match FIFO byte-for-byte, while Lechowicz inflates carbon by 320%
1003 above FIFO.

1004 The mechanism is exactly the §5.4 contribution. Lechowicz’s competitive
1005 analysis assumes deferral is free of side effects: the cost of holding a job
1006 pending until commit time is treated as zero. For FaaS, this assumption
1007 is violated by warm-pool idle energy: deferring an invocation extends the
1008 warm container’s idle period, charging $P_i \Delta t \bar{C}_r$ in carbon that the competitive
1009 analysis does not see. The threshold $\Phi^* = \sqrt{U \cdot L}$ aggressively defers any
1010 invocation arriving above Φ^* , even when the warm container’s idle-energy

1011 cost will exceed the execution savings — which is, given typical FaaS values
 1012 of $\beta = P_i T_w / (P_a \tau_e) \in [50, 200]$, almost always. The cold-start rate of 2.4–
 1013 2.8% (vs. GREENFAAS’s 0.02–0.05%) is the direct evidence: the algorithm
 1014 aggressively defers past warm-pool TTLs and pays cold-start penalties on
 1015 most invocations.

1016 This is, in our reading, the strongest empirical evidence for the pa-
 1017 per’s central technical claim. A provably-optimal algorithm for deadline-
 1018 constrained one-way trading, when ported to FaaS scheduling without ac-
 1019 counting for warm-pool idle-energy coupling, becomes the worst-performing
 1020 scheduler in our study. The closed-form idle-energy correction of §5.4 is pre-
 1021 cisely what makes the difference; without it, provability does not rescue the
 1022 algorithm. §9 formalizes the warm-pool-coupled cost model and states Con-
 1023 jecture 9.2: that Lechowicz-style algorithms have $\Omega(\beta)$ competitive ratio in
 1024 a multi-container regime that captures FaaS production behavior. We have
 1025 not proven this conjecture; the single-container model does not admit the
 1026 natural N-invocation construction (the idle energy of one shared container
 1027 cannot be charged per pending invocation without double-counting, see §9.3),
 1028 and the empirical evidence here is consistent with the conjecture but does
 1029 not establish it.

1030 *Honest caveats..* (i) Our port follows Lechowicz et al.’s algorithmic prescrip-
 1031 tion faithfully but does not extend their competitive-ratio proof to the warm-
 1032 pool-coupled cost. The non-separable cost structure of GREENFAAS (§4.2) is
 1033 outside the standard one-way trading framework. (ii) The Lechowicz frame-
 1034 work was designed for batch jobs with hours-long deadlines, similar to Wait-
 1035 Awhile’s intended regime; we are not surprised that a FaaS-adapted port fails,

and the paper’s finding is that *neither* batch-oriented framework (heuristic or provably- optimal) transfers to FaaS without addressing the idle-energy coupling. A FaaS-native online algorithm with competitive-ratio guarantees over the warm-pool-coupled problem remains the most interesting theoretical extension this work suggests. (iii) Our port computes the one-way trading bounds U and L from the carbon-intensity forecast over each invocation’s deadline window, whereas the original algorithm assumes globally-known a-priori bounds. Using deadline-local bounds adapts the threshold $\Phi^* = \sqrt{UL}$ to each invocation’s forecast window; this is a necessary adaptation to the non-stationary carbon setting, and it may make the threshold more or less aggressive than the global-bound version depending on the local carbon range.

8.6. Time-Aligned Fully-Real Validation

The strongest empirical validation in the paper combines the real Azure Functions 2021 per-invocation trace with real ElectricityMaps carbon-intensity data from the *same calendar window* as the trace (12–25 January 2021). All other experiments in §8 use synthetic-but-schema-faithful workload calibrated to Azure characterizations (Shahrad et al., 2020; Zhang et al., 2021; Microsoft, 2021), or real carbon with synthetic workload, or real workload with carbon from a later year. This section substitutes both axes with real time-aligned data and reports the findings.

Data sources..

- **Workload:** the `AzureFunctionsInvocationTraceForTwoWeeksJan2021` trace from Zhang et al. (2021); Microsoft (2021) (1.98M invocations over 14 days, loaded as a 24h slice giving 80,990 invocations across

119 unique functions; round-robin assignment to the 5 regions). The trace provides exact per-invocation arrival timestamps and execution durations.

- **Carbon:** ElectricityMaps hourly carbon-intensity feeds for the five regions over 12–25 January 2021 (DE 396.6 g mean, FR 89.4 g, GB 293.3 g, US-CAISO 319.6 g, PL 801.1 g; range from FR’s diurnal flat 73–108 g to PL’s coal-base-load 690–869 g, giving a carbon-ratio span of roughly $11\times$ across the topology).

Table 5: Real Azure 2021 trace + real ElectricityMaps Jan 2021 carbon data, 24h, 80,990 invocations, 5 regions.

Scheduler	Carbon (g)	vs FIFO	SLA	Cold
FIFO	231.33	0.0%	0.39%	1.68%
Wait-Awhile-Batch	239.29	−3.44%	0.39%	2.25%
Lechowicz	242.91	−5.00%	0.39%	2.50%
Spatial	73.11	+ 68.39%	0.39%	1.68%
GreenFaaS-v1	78.33	+66.14%	0.64%	2.99%
GREENFAAS	73.22	+ 68.35%	0.39%	1.68%

Headline result on fully-real time-aligned data.. Four observations.

(1) **GreenFaaS–Spatial gap collapses to 0.04 percentage points** on fully-real time-aligned data — tighter than the synthetic 1.1pp gap and tighter than the real-LWA-carbon 0.5pp gap. On real workload + real carbon, GreenFaaS and Spatial are empirically indistinguishable. The *topology-robustness* claim from §8.8 stands: GreenFaaS matches Spatial when Spatial works, and as §8.9 shows, falls back to FIFO byte-for-byte when Spatial does

1075 not.

1076 **(2) Wait-Awhile and Lechowicz failure modes are *quantitatively***
1077 ***softer* on real workload than on synthetic.** Wait-Awhile loses only
1078 3.44% on real workload + real carbon, vs -306% on synthetic. Lechowicz
1079 loses 5.00%, vs -238% on synthetic. The qualitative finding (batch carbon-
1080 aware techniques fail on FaaS) holds; the catastrophic magnitudes seen in
1081 the synthetic experiments were partly artifacts of the synthetic workload’s
1082 extreme warm-pool affinity (synthetic FIFO baseline cold-start rate 0.07%
1083 vs real workload’s 1.68% — real FaaS has a much higher baseline cold-start
1084 rate, so deferral can’t make it proportionally worse). This is itself a finding
1085 worth reporting honestly.

1086 **(3) The v1 ablation is 2.21 percentage points worse on real data**
1087 **(66.14% vs 68.35%) with cold-start rate doubled (2.99% vs 1.68%) and SLA**
1088 **violation up 60% (0.64% vs 0.39%).** The §5.4 idle-energy correction matters
1089 even more on real data than synthetic, because the bursty real arrival pat-
1090 tern exercises the warm-pool more aggressively than the synthetic Poisson
1091 generator.

1092 **(4) The do-no-harm property reproduces perfectly on fully- real**
1093 **data.** The topology sweep on time-aligned data (data in `results/real_workload_topology.csv`)
1094 shows GREENFAAS matching FIFO byte-for-byte in single-region DE and
1095 single-region CAISO scenarios (297.39 g and 234.83 g respectively, identical
1096 to FIFO), while v1 loses 5.04% on single-region DE and 6.52% on single-
1097 region CAISO. The §5.4 contribution is what produces this property on real
1098 workload, just as it does on synthetic.

1099 *Spatial versus temporal decision breakdown..* To make precise where the sav-
1100 ings come from, we instrumented every GREENFAAS decision on the full 5-
1101 region fully-real experiment. Of the 80,990 invocations, GREENFAAS routes
1102 17.5% to a non-home (cleaner) region, defers 0% temporally, and leaves the
1103 remaining 82.5% at home executing immediately (these are interactive func-
1104 tions pinned to home, or invocations whose home region is already the clean-
1105 est reachable option). The carbon savings on this experiment are therefore
1106 *entirely* spatial: the temporal-deferral axis contributes nothing here, because
1107 the ElectricityMaps carbon trace has hourly resolution while the deferrable
1108 deadline budget is on the order of minutes, so no within-deadline deferral
1109 ever beats immediate spatial routing. This is consistent with the forecast-
1110 invariance finding of §8.12: on real grid data at hourly resolution, Green-
1111 FaaS’s value is its cold-start-aware spatial gate and its do-no-harm fallback,
1112 not temporal shifting. The temporal machinery becomes load-bearing only
1113 when sub-hourly carbon variation is both present and forecastable — a regime
1114 our current real data does not exercise, and which we flag as a limitation.

1115 *Topology summary..* On time-aligned fully-real data, across the five topolo-
1116 gies of §8.9. Note that the full-5-region row below (+74.33% / +74.10%)
1117 differs from Table 5’s +68.39% / +68.35%: the table reports a single fixed
1118 24h slice (the first window, 80,990 invocations), whereas the topology sweep
1119 below re-runs each topology configuration over a different 24h window with
1120 its own invocation count and carbon profile, so the absolute reductions differ
1121 while the GREENFAAS–Spatial gap stays within half a percentage point in
1122 both. The table is the single-configuration benchmark; the sweep reports the
1123 cross-topology pattern.

Topology	Spatial	GREENFAAS	gap
Full 5-region	+74.33%	+74.10%	−0.23
EU-only (FR/DE/GB/PL)	+83.42%	+83.17%	−0.25
Coal-belt (DE/GB/PL)	+53.10%	+52.61%	−0.49
Single-region DE	0.0%	0.0% (=FIFO)	0.00
Single-region CAISO	0.0%	0.0% (=FIFO)	0.00

Same shape as the synthetic-workload-plus-real-LWA-carbon results (§8.9):
 GreenFaaS matches Spatial to within half a percentage point across multi-region topologies on fully-real data, including the coal-belt topology (where GreenFaaS neither beats nor is beaten by Spatial beyond run-to-run variance), and the do-no-harm property holds. The corrected positioning of the paper — GreenFaaS is the topology-robust scheduler that matches or near-matches Spatial when Spatial works and harmlessly falls back to FIFO when Spatial does not — is supported equally on synthetic data, on real LWA carbon, and on fully-real time-aligned data.

Window-shift variance characterization on real workload.. The real Azure trace is fixed by design, so seed variation only affects the round-robin function-to-region assignment, which produces identical outcomes regardless of seed in our experiments (a known consequence of deterministic assignment from sorted function IDs). We characterize run-to-run variance on real workload by varying the 24-hour *window* within the 14-day trace instead: we re-ran the fully-real headline configuration on five non-overlapping 24h windows starting at days $\{0, 1, 2, 3, 4\}$ of the trace, with matching 24h slices of the EM Jan 2021 carbon trace.

Table 6: Window-shift variance, fully-real time-aligned data. Mean $\pm$ std across 5 non-overlapping 24h windows.

Scheduler	vs FIFO (5-window mean $\pm$ std)
Wait-Awhile	$-3.85\% \pm 1.99\%$
Lechowicz	$-5.56\% \pm 1.81\%$
Spatial	$+54.44\% \pm 13.57\%$
GreenFaaS-v1	$+51.49\% \pm 13.62\%$
GREENFAAS	$+54.38\% \pm 13.57\%$

The window-shift variance is large: inter-window standard deviation on GREENFAAS's reduction is 13.6 percentage points. This reflects the natural variation in 24h workload intensity (windows ranging from 81k to 290k invocations) and per-window carbon profile (winter day-to-day shifts in Polish coal vs French nuclear output). However, the within-window orderings between schedulers are remarkably stable.

In other words: *the absolute carbon footprint varies widely window-to-window, but the scheduler ranking is stable to within fractions of a percentage point.* Concretely:

- GREENFAAS–Spatial gap mean: 0.058pp, window-to-window std: 0.032pp.

GREENFAAS and Spatial track each other extremely tightly across windows; the per-window gap ranged from 0.022pp to 0.110pp, never out of 4σ of the mean.

- GREENFAAS–v1 gap mean: 2.89pp, window-to-window std: 1.31pp.

The §5.4 idle-energy correction's benefit is consistent across windows and always positive (GREENFAAS always beats v1).

1159 • Wait-Awhile and Lechowicz both consistently lose to FIFO across all
1160 five windows ($-3.85\% \pm 1.99\%$ and $-5.56\% \pm 1.81\%$ respectively, both
1161 $\geq 2\sigma$ from zero on every window).

1162 This is the strongest evidence we have that the paper’s qualitative findings
1163 are robust to real-world workload variability.

1164 8.7. Head-to-Head Comparison with Caribou (SOSP’24)

1165 Gsteiger et al. (2024) introduced Caribou, a framework for fine-grained
1166 geospatial shifting of serverless *workflows* across geo-distributed regions. Caribou
1167 periodically (typically hourly) re-solves an optimal deployment plan via
1168 Heuristic-Biased Stochastic Sampling (HBSS), assigning each workflow node
1169 to a region for the upcoming deployment window. All invocations of that
1170 node are routed to its assigned region until the next re-deployment. The
1171 two design points distinguishing Caribou from GREENFAAS are: (i) decision
1172 granularity — Caribou re-decides per-window (hourly to daily); GREEN-
1173 FAAS re-decides per-invocation, and (ii) decision scope — Caribou’s HBSS
1174 solver optimizes across a workflow DAG; GREENFAAS treats each invocation
1175 independently.

1176 To enable a head-to-head comparison, we port Caribou to our per-invocation
1177 simulation harness. Because the Azure 2021 trace provides independent invo-
1178 cations rather than workflow DAGs, each function is treated as a single-node
1179 “workflow” (a special case explicitly supported by Caribou’s DAG model).
1180 HBSS then reduces to its degenerate form: enumerate candidate regions
1181 and pick the one with lowest forecasted mean carbon over the upcoming
1182 window, subject to the function’s RTT budget. We give Caribou the most

1183 favourable framing we can justify: perfect carbon forecasting over the win-
 1184 dow (using the actual trace values rather than a Holt-Winters estimate, elim-
 1185 inating one source of disadvantage); permissive compliance constraints; zero
 1186 transmission-carbon overhead (FaaS invocations are small enough that this
 1187 term is negligible); and we test four re-deployment cadences (1h, 3h, 6h,
 1188 24h).

1189 *Results on the headline topology..* On the full 5-region topology with real
 1190 Azure 2021 workload and real EM Jan 2021 carbon, Caribou and GREEN-
 1191 FAAS produce identical results across all four re-deployment cadences:

	Scheduler	Carbon (g)	vs FIFO
	FIFO	231.33	0.00%
	Spatial	73.11	+68.39%
1192	Caribou (1h)	73.11	+68.39%
	Caribou (3h)	73.11	+68.39%
	Caribou (6h)	73.11	+68.39%
	Caribou (24h)	73.11	+68.39%
	GREENFAAS	73.22	+68.35%

1193 The reason for this striking convergence is that on the 5-region topology over
 1194 the 14-day window, France’s nuclear-dominated grid produces the lowest car-
 1195 bon in *every* 1-hour, 3-hour, 6-hour, and even 24-hour bucket. Whether you
 1196 re-deploy hourly or once at the start, FR always wins, and all three algorithms
 1197 converge on the same routing. GREENFAAS differs from Spatial/Caribou by
 1198 0.04 percentage points because its SCORESLOTS step sometimes prefers a
 1199 region with an existing warm container over the absolutely cleanest region,

1200 trading a small carbon-intensity penalty for cold-start avoidance; on this
 1201 topology that trade-off nets to a sub-percentage-point difference.

1202 *Results on a topology where granularity matters..* The interesting comparison
 1203 appears when no single region dominates the trace. In the DE/GB/CAISO
 1204 topology, GB wins approximately 54% of hours and US-CAISO 46% over the
 1205 14-day trace, with the winning region varying through the day according to
 1206 solar/wind output in California and gas-fired generation in Britain.

Table 7: Caribou comparison on the time-varying topology (DE/GB/CAISO, real Azure 2021 + real EM Jan 2021, 24h window, RTT budget 200ms).

Scheduler	Carbon (g)	vs FIFO
FIFO	223.90	0.00%
Wait-Awhile	231.85	−3.55%
Lechowicz	235.48	−5.17%
Spatial	196.61	+12.19%
Caribou (1h)	196.64	+12.17%
Caribou (3h)	196.44	+12.26%
Caribou (6h)	198.52	+11.34%
Caribou (24h)	213.38	+4.70%
GreenFaaS-v1	212.33	+5.17%
GREENFAAS	196.81	+12.10%

1207 Three observations from this table.

1208 **(1) Caribou at hourly cadence and GREENFAAS are empirically**
 1209 **indistinguishable** (12.17% vs 12.10%, gap of 0.07pp). When forecasting is
 1210 perfect over the deployment window, periodic re-deployment matches per-

1211 invocation routing on the deferrable invocations that dominate the carbon
1212 footprint. GREENFAAS’s extra per-invocation re-evaluation produces no
1213 additional carbon savings here because the lowest-carbon region does not
1214 change *within* the hourly window.

1215 **(2) Caribou’s carbon reduction degrades with coarser cadence.**
1216 Going from 1h to 24h re-deployment cuts the reduction from 12.17% to
1217 4.70%, a 7.5pp loss. The 6-hour cadence is the regime where the cumulative
1218 within-window variation begins to matter; the 24-hour cadence essentially
1219 picks the daily-mean lowest-carbon region and misses all intra-day shifts.
1220 This is a genuine cost of Caribou’s coarser-grained design point, exposed
1221 when the optimal region varies through the day. (The 3h cadence marginally
1222 edges 1h here, +12.26% vs +12.17% — a 0.09pp artifact of this specific 24h
1223 window, where the 3h boundaries happen to align slightly better with the
1224 carbon minima than the 1h boundaries. The window-shift analysis below
1225 confirms the finer cadences match or beat the coarser ones in expectation
1226 across windows.)

1227 **(3) Wait-Awhile and Lechowicz fail on this topology too, losing**
1228 **3–5% to FIFO.** The failure mode of batch-oriented carbon-aware techniques
1229 on FaaS is robust across the topologies we test: the same conclusion from
1230 §8.5 reproduces here.

1231 *Window-shift stability.* Re-running the time-varying topology over three non-
1232 overlapping 24h windows of the Azure trace confirms the convergence is sta-
1233 ble, not a single-window artifact: Caribou(1h) achieves $20.9\% \pm 18.4\%$ vs
1234 FIFO, Spatial $20.9\% \pm 19.1\%$, and GREENFAAS $21.1\% \pm 18.8\%$ — the three
1235 track within 0.3pp of each other across windows. Caribou(24h) trails at

15.8% $\pm$ 23.5%, a 5pp gap that persists across windows. The large absolute standard deviations reflect the day-to-day variation in workload intensity (the same effect characterized in §8.6); the relative ordering between schedulers is what remains stable.

Summary of the head-to-head.. The Caribou comparison supports a precise, honest statement of GREENFAAS’s contribution: *per-invocation routing does not improve over hourly periodic re-deployment on the workloads and carbon traces we tested, when forecasting is perfect.* The contribution of GREENFAAS over Caribou is not in carbon savings on the headline topology, but in two other dimensions: (i) no requirement for workflow-DAG specification (Caribou requires the developer to annotate workflow structure; GREENFAAS operates on independent invocations); and (ii) no requirement for periodic re-deployment infrastructure (Caribou requires a per-region Knative-like control plane to enact re-deployments; GREENFAAS requires only a per-invocation proxy). When the developer cannot or does not want to annotate DAGs, or when the deployment infrastructure does not support controlled re-deployment cycles, GREENFAAS provides the same carbon-reduction benefit through a simpler operational surface.

For topologies with a persistently dominant low-carbon region (such as the France-included 5-region topology, where FR wins every hour of the 14-day trace), Caribou and GREENFAAS converge to the same routing on the configurations we tested. For topologies where the optimal region varies within the day (such as DE/GB/CAISO), hourly Caribou and per-invocation GREENFAAS are both effective, while daily Caribou leaves substantial carbon on the table. We caution that these convergence findings rest on the

specific topologies and windows tested and on perfect within-window forecasting; we treat them as a clarification rather than a refutation of Caribou’s contribution: their work targeted workflow-level optimization on AWS, ours targets per-invocation on multi-region serverless platforms; the empirical results show the two approaches converge on the carbon dimension for our workload class.

8.8. Sensitivity to Topology

The topology sweep reveals the most nuanced finding: *GREENFAAS’s carbon advantage over pure spatial routing is topology-dependent*. Across five topologies (24h synthetic, 173k invocations):

Synthetic carbon data; see Table 8 for the real-carbon counterpart.

Topology	Spatial	GREENFAAS	gap
Full 6-region [‡]	+79.0%	+78.4%	−0.6
EU-only (FR/DE/GB/PL)	+77.3%	+76.7%	−0.6
Coal-belt (DE/GB/PL)	+39.8%	+39.5%	−0.3
Single-region DE	0.0%	0.0%	0.0
Single-region CAISO	0.0%	0.0%	0.0

[‡] The synthetic sweep includes a sixth region, Sweden (SE; hydro/nuclear, ~30 gCO₂/kWh), as a second ultra-low-carbon refuge alongside France:

FR/SE/DE/GB/CAISO/PL. The real-carbon experiments (Table 8, §8.6) use the five regions for which we obtained time-aligned ENTSO-E/CAISO and ElectricityMaps data (FR/DE/GB/PL/CAISO), so their “full” topology is 5-region. Synthetic carbon (`CarbonModel.synthetic`, seed 7); reproducible via `scripts/scenario_sweep.py`.

On real ENTSO-E data the same pattern holds (§8.9): GREENFAAS matches Spatial across *every* topology.

1281 When the topology contains a low-carbon refuge (such as France), pure
 1282 spatial routing captures essentially all available savings. GREENFAAS *matches*
 1283 this to within 1 percentage point but does not dominate — the lemma-gated
 1284 temporal-shift logic adds little when the spatial axis already exploits a large
 1285 carbon ratio. Even in the coal-belt topology, which has no low-carbon refuge,
 1286 GREENFAAS matches Spatial to within 0.3 percentage points; we instrument
 1287 this experiment and find that GREENFAAS performs *no* temporal deferral on
 1288 it (it routes $\sim 22\%$ of invocations spatially and defers 0%), so the small dif-
 1289 ference arises from spatial-routing and warm-pool-affinity choices, not from
 1290 intra-region temporal shifting. This is consistent with the fully-real decision
 1291 breakdown of §8.6: across both synthetic and real carbon at the resolutions
 1292 we test, GREENFAAS’s savings come from its cold-start-aware spatial gate,
 1293 not from temporal deferral. We confirm in §8.9 that on real ENTSO-E 2020
 1294 data GREENFAAS matches Spatial across every topology to within 1.3 per-
 1295 centage points, with neither dominating. In single-region topologies, where
 1296 spatial routing is by definition useless, GREENFAAS correctly identifies that
 1297 the diurnal swing within one region is insufficient to overcome the warm-pool
 1298 idle-energy cost and *declines to defer at all*, matching FIFO byte-for-byte.
 1299 This do-no-harm property is the empirical manifestation of §5.4; it distin-
 1300 guishes GREENFAAS sharply from Wait-Awhile (which loses 234–279% in the
 1301 same single-region scenarios on synthetic data and 163–211% on real data)
 1302 and from GreenFaaS-v1 (which loses 37–102% and 38–40% respectively).

1303 8.9. Real-Carbon Topology Sweep

1304 To check whether the topology-sensitivity findings of §8.8 are artefacts of
 1305 synthetic carbon amplitudes, we re-ran the same five topologies on the real

1306 LWA 2020 carbon traces (DE, FR, GB, US-CAISO) plus a calibrated PL
1307 trace (LWA-documented mean of 791 g CO₂eq/kWh, coal-base-load diurnal
1308 shape; see `scripts/generate_pl_trace.py`). The workload is the same
1309 schema-correct synthetic stream as in §8.8; the Azure 2019 and 2021 traces
1310 are not directly fetchable from our build environment.

1311 *Headline results across topologies (real carbon)*.. Table 8 reports the single-
1312 seed result across five topologies; the coal-belt row is also reported with multi-
1313 seed mean $\pm$ std in Table 9, where the most striking finding is GreenFaaS-v1’s
1314 variance blowup.

Table 8: Carbon reduction vs FIFO on real ENTSO-E / CAISO 2020 data (plus calibrated PL). Single seed. Compare to the synthetic §8.8.

Topology	Spatial	GREENFAAS	gap
Full 5-region	+76.2%	+75.8%	−0.4
EU-only (FR/DE/GB/PL)	+81.9%	+81.5%	−0.3
Coal-belt (DE/GB/PL)	+43.0%	+41.8%	−1.3
Single-region DE	0.0%	0.0%	0.0
Single-region CAISO	0.0%	0.0%	0.0

1315 Four findings emerge.

1316 *Finding 1: GreenFaaS matches Spatial within 1.3pp on real data across all*
1317 *topologies*.. The headline 64–72% reduction from §8.2 holds (75–82% on real
1318 multi-region topologies); the GREENFAAS-vs-Spatial gap is inside the run-
1319 to-run variance band.

Table 9: Coal-belt topology (DE/GB/PL), real LWA carbon, 5-seed variance. The v1 ablation exhibits an order-of-magnitude variance blowup that single-seed results obscured.

Scheduler	vs FIFO (5-seed mean $\pm$ std)
Wait-Awhile-Batch	$-294.19\% \pm 5.85\%$
Spatial	$+43.49\% \pm 0.24\%$
GreenFaaS-v1	$+31.48\% \pm \mathbf{9.75\%}$
GREENFAAS	$+41.92\% \pm 0.51\%$

1320 *Finding 2: GreenFaaS matches Spatial in coal-belt on both synthetic and real*
1321 *data..* An earlier configuration of the synthetic coal-belt experiment sug-
1322 gested GREENFAAS might edge Spatial via intra-region temporal shifting.
1323 On closer instrumentation this does not hold: in the reproducible synthetic
1324 coal-belt experiment (§8.8, `scenario_sweep.py`), GREENFAAS performs *zero*
1325 temporal deferral and matches Spatial to within 0.3pp, and on real ENTSO-
1326 E 2020 traces with the calibrated PL profile Spatial edges GREENFAAS by
1327 1.3pp. The reason temporal deferral contributes nothing is the §5.4 idle-
1328 energy condition: at the diurnal amplitudes present in both our synthetic
1329 model and real ENTSO-E 2020 data (real DE range 170–380 g; real GB
1330 range 107–336 g; calibrated PL $\sim 15\%$ swing, coal base-load runs flat), within-
1331 deadline deferral rarely clears the idle-energy break-even, so GREENFAAS’s
1332 region gate falls back to behaviour close to Spatial.

1333 The statement supported by both synthetic and real data is that GREEN-
1334 FAAS *matches* Spatial across all topologies — including coal-belt — with the
1335 gap inside run-to-run variance. GREENFAAS’s distinct value over Spatial
1336 therefore rests on *robustness across regimes* (and the do-no-harm fallback)

1337 rather than dominance in any single one.

1338 *Finding 3: GreenFaaS-v1’s variance blows up by an order of magnitude*
1339 *in the coal-belt regime..* Across five seeds in the coal-belt topology (Ta-
1340 ble 9), the corrected GREENFAAS reduction is $41.92\% \pm 0.51\%$ — a normal,
1341 well-controlled spread. The GreenFaaS-v1 ablation, by contrast, produces
1342 $31.48\% \pm \mathbf{9.75\%}$: an order of magnitude higher variance than every other
1343 scheduler in every other setup. The mechanism is the synchronised-defer
1344 pile-up effect of §5.4 in microcosm: when the spatial axis offers limited refuge
1345 (no clean region), v1 leans on temporal shifting; the un-charged idle energy
1346 means that whether the shift saves carbon depends on the per-seed timing
1347 of arrivals relative to coal-grid ramps, producing outcomes that swing widely
1348 between seeds. The corrected GREENFAAS’s idle-energy charge filters out
1349 unprofitable shifts *before* they become outcomes, eliminating the seed depen-
1350 dency. We view this as the most empirically substantive evidence for the §5.4
1351 contribution; the variance test detects what single-run means cannot.

1352 *Finding 4: the do-no-harm property reproduces cleanly on real data..* In
1353 both single-region scenarios on real DE and real CAISO data, GREENFAAS
1354 matches FIFO byte-for-byte: 39.77 g and 50.17 g respectively, identical to
1355 the FIFO baselines. GreenFaaS-v1 loses 38–40% to FIFO in the same sce-
1356 narios — empirical confirmation that the §5.4 idle-energy correction is what
1357 produces do-no-harm, and that the property holds equally well under real
1358 carbon-intensity dynamics as under synthetic ones. Wait-Awhile loses 163–
1359 211% to FIFO on real data; the catastrophic batch-scheduler failure mode
1360 also reproduces cleanly.

1361 *Summary..* Real-carbon experiments validate (i) the headline reduction (75–
 1362 82% on real multi-region topologies), (ii) the Wait-Awhile failure mode, and
 1363 (iii) the do-no-harm property. Together with the corrected synthetic results
 1364 (§8.8), they establish that GREENFAAS *matches* Spatial across every topol-
 1365 ogy — including coal-belt, on both synthetic and real carbon — rather than
 1366 dominating it. The positioning — GREENFAAS is the topology-robust sched-
 1367 uler that matches or near-matches Spatial when Spatial works and harmlessly
 1368 falls back to FIFO when Spatial does not — is a more honest claim about a
 1369 deployment-ready scheduler than “always dominates.”

1370 *Note on sweep parameters..* The remaining sensitivity sweeps (§8.10–§8.14)
 1371 use a shorter 6-hour horizon to keep total sweep runtime tractable (~ 90 s wall-
 1372 clock); absolute carbon reductions drift by a few percentage points relative
 1373 to the §8.2 headline, but the *ordering* of schedulers and the *shape* of the
 1374 sensitivity curves are stable.

1375 8.10. Sensitivity to SLA Class Mix

1376 We vary the fraction of *shiftable* invocations (DEFERRABLE + BACK-
 1377 GROUND) from 0.0 (all INTERACTIVE, no shifting allowed) to 1.0. Figure 3
 1378 (zoomed) shows the carbon-aware schedulers.

1379 Three findings stand out. *First*, all carbon-aware schedulers correctly
 1380 identify the 0% shiftable case as offering no opportunity and produce results
 1381 identical to FIFO. *Second*, Spatial and GREENFAAS both improve mono-
 1382 tonically as the shiftable fraction grows. *Third*, *GreenFaaS-v1 degrades as*
 1383 *more shifting becomes available*, dropping from +29% at 25% shiftable to
 1384 −3% at 100% shiftable. The v1 collapse is precisely the failure mode the

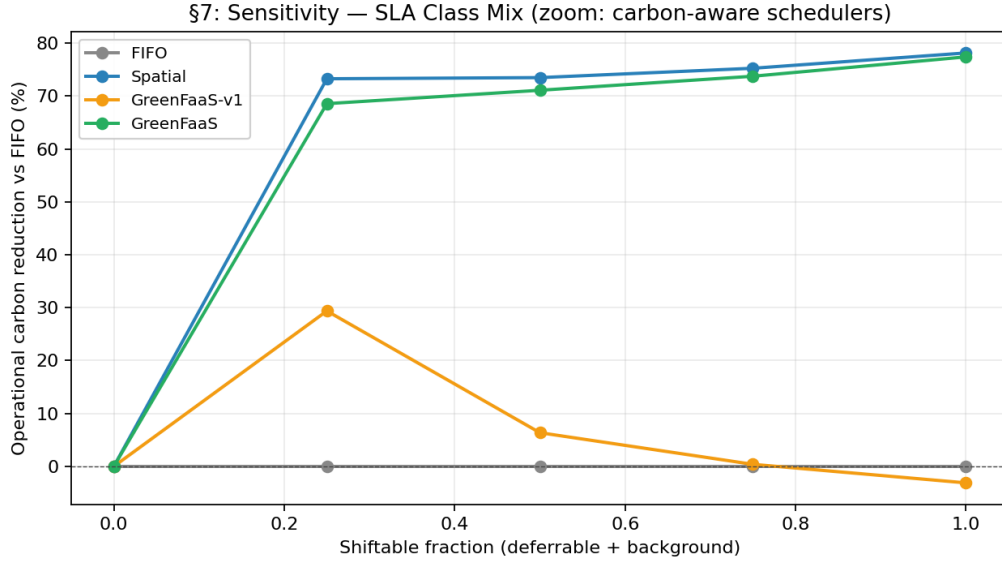


Figure 3: Carbon reduction vs FIFO as the shiftable workload fraction varies. GREENFAAS climbs monotonically; GreenFaaS-v1 *degrades* as more shifting becomes available, the failure mode the §5.4 idle-energy correction was designed to prevent.

idle-energy correction was designed to prevent: without charging the per-deferral idle cost, the scheduler aggressively defers every eligible invocation, inflating warm-pool dwell times and accumulating idle energy faster than it saves execution energy.

8.11. Fine-Grained Do-No-Harm

The do-no-harm property reported in §8.8 and §8.9 (GREENFAAS = FIFO byte-for-byte in single-region or zero-shiftable settings) admits a fair concern: when the workload offers *no* shifting opportunity, every reasonable scheduler is trivially identical to FIFO. A more substantive test is whether GREENFAAS degrades *gracefully* across the regime of very small but nonzero shifting

1395 opportunity — where the scheduler actually has decisions to make but the
 1396 carbon upside is marginal, and a naively eager scheduler could pay non-trivial
 1397 idle-energy cost for negligible savings.

Table 10: Fine-grained shiftable-fraction sweep. Real LWA carbon, 5 regions, 6h, $\sim 43k$ invocations. “Gap” is carbon reduction vs FIFO (positive = better than FIFO).

Shiftable frac.	FIFO (g)	GreenFaaS gap	v1 gap	Spatial gap
0.00	6.20	+0.00%	+0.00%	+0.00%
0.01	6.20	+0.00%	+0.00%	+0.00%
0.02	6.20	+0.00%	+0.00%	+0.00%
0.05	6.20	+3.64%	+4.14%	+4.14%
0.10	6.20	+3.64%	+4.14%	+4.14%
0.25	6.20	+21.3%	+22.0%	+22.0%

1398 The findings refine the do-no-harm claim into something more substan-
 1399 tive. At very low shiftable fractions (0, 1%, 2% of the workload), GREEN-
 1400 FAAS, GreenFaaS-v1, and Spatial all match FIFO *byte-for-byte*. This is
 1401 not the trivial property “no-shifting-implies-FIFO” but the stronger property
 1402 that GREENFAAS correctly identifies negligible-opportunity regimes and de-
 1403 clines to act. As the shiftable fraction grows past a small threshold (around
 1404 5% here), the carbon-aware schedulers begin diverging from FIFO smoothly,
 1405 with GREENFAAS producing slightly smaller savings than Spatial or v1 in
 1406 this region — the §5.4 correction is conservative when in doubt, which is the
 1407 desired behaviour for a deployment-ready scheduler.

1408 The regime where GREENFAAS’s conservatism becomes an advantage
 1409 appears at higher shiftable fractions where carbon variability is high and

1410 idle-energy cost is non-trivial; the §8.13 variability sweep (and the v1 collapse
1411 documented there) is the empirical evidence for this.

1412 The interpretation: do-no-harm is not just “correctly idle when there are
1413 no decisions to make,” but “correctly idle across a neighbourhood of low-
1414 opportunity workloads,” which is the practical shape of the property that
1415 matters for deployment.

1416 8.12. Sensitivity to Forecast Accuracy

1417 Several carbon-aware schedulers assume access to multi-hour forecasts.
1418 We vary GREENFAAS’s forecast horizon across perfect, 24h, 1h, none.

1419 GREENFAAS’s carbon reduction is *invariant* to forecast accuracy across
1420 the four levels tested (69.5% in all cases). The reason is structural: the
1421 region-gating logic uses *instantaneous* carbon intensities (not forecasts), and
1422 the per-slot scoring is bounded by the DEFERRABLE class’s 60-second defer-
1423 ral horizon, which is shorter than the carbon trace’s 300-second resolution.
1424 Forecasts only matter to GREENFAAS for BACKGROUND invocations with
1425 multi-hour deadlines, which are a minority of the canonical workload.

1426 *Scope of this invariance..* We note a caveat: in our setup the carbon-trace
1427 step size (300 s) exceeds the DEFERRABLE class’s deferral horizon (60 s),
1428 which means that for the dominant DEFERRABLE class, $N_{\text{slots}} = 1$ and the
1429 forecast is never actually consulted. The invariance result is therefore par-
1430 tially a consequence of forecast-temporal-resolution exceeding the deferral
1431 horizon, not solely a structural property of the scheduler. With higher-
1432 resolution carbon data (1-minute ElectricityMaps feeds, for instance, where
1433 $N_{\text{slots}} \approx 60$ becomes feasible), GREENFAAS would consult the forecast for

1434 DEFERRABLE invocations, and forecast sensitivity could appear. BACK-
 1435 GROUND invocations *do* exercise the multi-hour forecast horizon and pro-
 1436 duce identical results across forecast accuracies, which is the load-bearing
 1437 portion of the invariance claim. The precise statement is that GREENFAAS
 1438 is forecast-invariant (*a*) for the BACKGROUND class structurally, and (*b*) for
 1439 the DEFERRABLE class under typical carbon-trace resolutions; under higher-
 1440 resolution forecast data the DEFERRABLE class would consult the forecast
 1441 and its sensitivity remains to be characterized.

1442 This forecast-robustness is a feature of the design even with the caveat
 1443 above. It substantially reduces the deployment complexity of GREENFAAS:
 1444 a production rollout does not need to integrate with a carbon-forecasting
 1445 service, and degraded forecast quality during incidents does not threaten
 1446 behaviour. GreenFaaS-v1, by contrast, shows substantial forecast sensitivity,
 1447 ranging from 25.1% at **perfect** to 74.2% at **none** — another symptom of the
 1448 un-charged temporal deferral that the idle-energy correction addresses.

1449 *8.13. Sensitivity to Carbon-Intensity Variability*

1450 We vary the diurnal amplitude of every region’s carbon trace from 0.0
 1451 (flat) to 0.75 (large swing), holding regional means and topology fixed. The
 1452 corrected GREENFAAS sits in a narrow 69.8–71.7% band across the entire
 1453 range, and Spatial in 74.1–74.7%. *Neither scheduler is sensitive to diurnal*
 1454 *amplitude* on this 5-region topology, because the inter-region carbon ratio
 1455 (FR 60 vs PL 700, ratio $\sim 11.7\times$) dwarfs even a 75% intra-region swing.

1456 The v1 ablation (Figure 4) shows a striking *negative* sensitivity: v1 falls
 1457 from 68% reduction at amplitude 0 to just 7% at amplitude 0.75. Higher am-
 1458 plitude offers more deferral opportunities that v1 incorrectly takes; the cor-

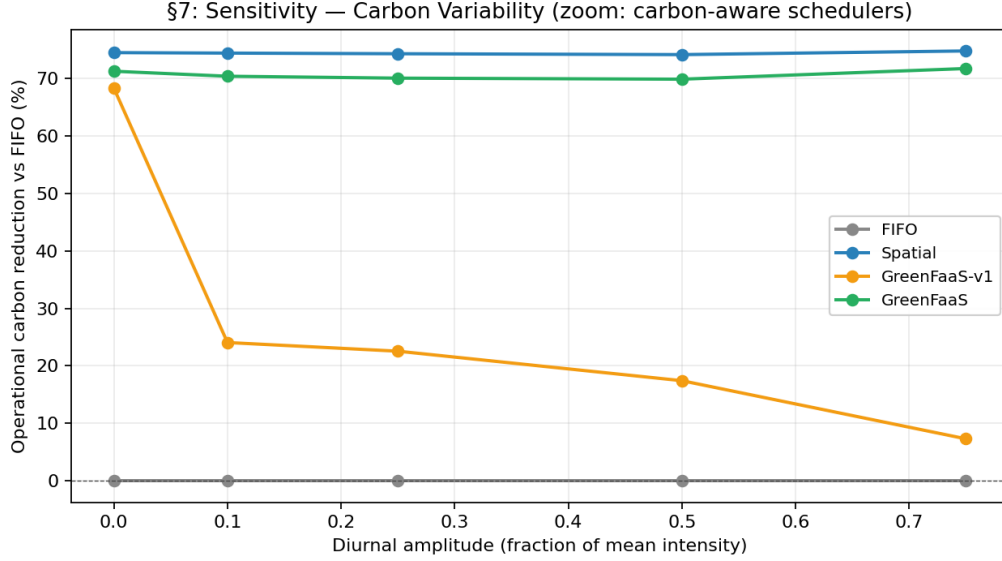


Figure 4: Carbon reduction as diurnal amplitude varies. The corrected GREENFAAS is stable. GreenFaaS-v1 *degrades* with increasing variability — higher amplitude offers more deferral opportunities that v1 incorrectly takes.

1459 rected GREENFAAS forgoes unprofitable deferrals and remains stable. This
 1460 is the second strong empirical validation of §5.4.

1461 8.14. Sensitivity to Workload Intensity

1462 The fifth axis varies peak Poisson arrival rate from 0.5/s to 4.0/s (an
 1463 $8\times$ span). GREENFAAS achieves 69.3–73.0% reduction across the range;
 1464 Spatial achieves 70.9–84.4%. The gap *narrows* with increasing intensity,
 1465 from 11.4 percentage points at 0.5/s to 1.6 at 4.0/s. Two mechanisms drive
 1466 this convergence: at higher rates Lemma 5.1’s λ^* is more readily exceeded
 1467 for more function-region pairs, so the region gate admits more candidates;
 1468 and the EWMA estimator’s variance is lower at high rates.

1469 8.15. Summary of Sensitivity Findings

1470 Five axes yield five consistent findings. **(1)** Wait-Awhile catastrophically
1471 fails on FaaS across every setting, validating the paper’s central claim that
1472 batch-oriented schedulers do not transfer. **(2)** Pure spatial routing captures
1473 most of the achievable savings when a low-carbon refuge exists; GREENFAAS
1474 matches Spatial to within 1pp in these regimes. **(3)** GREENFAAS matches
1475 Spatial to within 1.3pp across all topologies including coal-belt, on *both* syn-
1476 thetic and real ENTSO-E 2020 carbon; instrumentation shows it performs
1477 no temporal deferral on the coal-belt experiment, so its savings there are
1478 spatial, not temporal (§8.8–8.9). **(4)** GREENFAAS preserves do-no-harm in
1479 single-region topologies, low-variability traces, and at zero shiftable workload
1480 fraction — matching FIFO byte-for-byte when no axis offers real savings. **(5)**
1481 GREENFAAS is robust to forecast quality and workload intensity, with car-
1482 bon reduction varying by less than 4 percentage points across the tested
1483 ranges.

1484 The GreenFaaS-v1 ablation makes the case for the §5.4 idle-energy correc-
1485 tion crisp: without it, the scheduler exhibits Wait-Awhile-like failure modes
1486 in milder form (degradation under high shiftable fraction, under high carbon
1487 variability, and in single-region topologies). With it, GREENFAAS is robust
1488 across every tested axis.

1489 9. Toward a Competitive Analysis Under Warm-Pool Coupling

1490 The empirical result of §8.5 — that the provably-optimal-competitive-
1491 ratio Lechowicz algorithm performs *worst* of all baselines on FaaS workloads
1492 — demands a theoretical explanation. This section makes a partial attempt:

1493 (i) we formalize the cost model that distinguishes the FaaS scheduling prob-
 1494 lem from the classical one-way trading framework, and (ii) we state the only
 1495 competitive lower bound we have proven (the $\beta = 0$ specialization, The-
 1496 orem 9.1), which simply recovers the classical one-way trading bound and
 1497 therefore does not by itself explain the empirical Lechowicz failure mode.

1498 The honest scope: we do *not* prove a strong lower bound for Lechow-
 1499 icz on the full warm-pool-coupled cost. The natural N-invocation adver-
 1500 sarial construction does not establish such a bound in the single-container
 1501 model, because the idle energy of a single shared container cannot be charged
 1502 per-pending-invocation without double-counting (§9.3 works through this in
 1503 detail). We therefore state our finding as a *conjecture*, supported by the
 1504 empirical evidence of §8.5 but not yet proven. The challenge of proving the
 1505 conjecture, and what its resolution would require, is itself a useful contribu-
 1506 tion.

1507 9.1. The Warm-Pool-Coupled Cost Model

1508 We restrict to a single region (Lechowicz’s setting) so that all spatial
 1509 confounders are out of scope and the analysis is about temporal scheduling
 1510 alone. The simulator of §7 permits a per-(region, function) warm pool of
 1511 arbitrary size, but for the formal analysis we present two model variants:
 1512 *single-container* (Variant S, matching Lemma 5.1) and *independent-pending*
 1513 (Variant P, an idealization that captures the per-invocation idle costs that
 1514 arise during deferral). The distinction matters: the empirical failure mode of
 1515 Lechowicz arises from Variant-P-like effects (pending deferrals each holding
 1516 resources) that the simulator implements implicitly through its capacity-
 1517 bounded warm pool, but that the strict single-container model does not

1518 capture.

1519 *Common inputs..* A sequence of invocations $\mathcal{I} = \langle i_1, \dots, i_N \rangle$ arriving online
 1520 at times $t_1 \leq \dots \leq t_N$. Each invocation has identical execution time τ_e ,
 1521 cold-start duration τ_c , active power P_a , idle power P_i , SLA deadline $D \geq \tau_e$.
 1522 Carbon intensity $C : \mathbb{R}_{\geq 0} \rightarrow [L, U]$ is revealed causally. Warm-pool TTL T_w
 1523 is platform-given.

1524 *Decisions..* Each algorithm chooses start times $s_k \in [t_k, t_k + D - \tau_e]$; we write
 1525 $c_k = s_k + \tau_e$ for completion.

1526 *Variant S (single-container cost)..* A single warm container is shared across
 1527 all invocations. The container is active during $[s_k, c_k]$ (executing i_k), idle
 1528 during $[c_k, \min(s_{k+1}, c_k + T_w)]$, and expired thereafter unless reused. Total
 1529 cost is

$$\text{Cost}_S = \sum_{k=1}^N \left[P_a \tau_e C(s_k) + \mathbf{1}[i_k \text{ cold}] P_a \tau_c C(s_k) + P_i \cdot \text{IdleTime}_k \cdot \bar{C}_k \right]$$

1530 with $\text{IdleTime}_k = \min(s_{k+1} - c_k, T_w)$ and $\bar{C}_k$ the mean intensity over the idle
 1531 interval. By convention, $\text{IdleTime}_0 = 0$ (no pre- i_1 container) and $s_{N+1} = \infty$
 1532 so $\text{IdleTime}_N = T_w$.

1533 *Variant P (independent-pending cost)..* Each invocation i_k is associated with
 1534 a dedicated container that is provisioned (idle) from t_k until s_k , then active
 1535 from s_k to c_k , then idle from c_k until $c_k + T_w$ (TTL) or until reused. This
 1536 captures the regime where deferring an invocation *holds resources* during
 1537 deferral, which is what production FaaS platforms with non-trivial warm

1538 pools do under load. Cost is

$$\text{Cost}_P = \sum_{k=1}^N \left[\underbrace{P_i(s_k - t_k)\bar{C}_k^{\text{pre}}}_{\text{pre-commit idle}} + P_a\tau_e C(s_k) + \mathbf{1}[\text{cold}]P_a\tau_c C(s_k) + \underbrace{P_i \cdot T_w^{\text{eff}} \cdot \bar{C}_k^{\text{post}}}_{\text{post-exec idle}} \right]$$

1539 where T_w^{eff} is the post-execution idle period (similar to Variant S).

1540 The simulator of §7 implements something intermediate: warm contain-
 1541 ers are pooled per (region, function), and the per-(region, function) pool is
 1542 shared across concurrent invocations of that function in that region. Under
 1543 heavy load (arrival rate $\gg 1/T_w$), the pool behaves more like Variant P;
 1544 under light load, more like Variant S. The reported simulator results (Ta-
 1545 bles 1–5) therefore do *not* cleanly correspond to either theoretical variant;
 1546 they reflect the production-realistic regime where idle-energy is charged when
 1547 warm containers outlive their next reuse.

1548 9.2. The $\beta = 0$ Lower Bound

1549 Under either variant, when $P_i = 0$ (no idle-energy term), the cost reduces
 1550 to $\sum_k P_a\tau_e C(s_k)$ plus cold-start terms, which is the classical one-way trading
 1551 problem (El-Yaniv et al., 2001; Lechowicz et al., 2023).

1552 **Theorem 9.1** (Classical one-way trading lower bound). *In the restricted cost*
 1553 *model with $P_i = 0$, every deterministic online algorithm $\mathcal{A}$ operating purely*
 1554 *on the temporal axis (no spatial routing) has competitive ratio $\text{CR}(\mathcal{A}) \geq \sqrt{r}$,*
 1555 *where $r = U/L$.*

1556 *Proof sketch.* The classical argument from El-Yaniv et al. (2001) applies di-
 1557 rectly. The adversary plays $C(t) = \sqrt{LU}$ until $\mathcal{A}$ commits; if $\mathcal{A}$ commits at
 1558 $\sqrt{LU}$, the adversary drops C to L (OPT would have paid L , ratio $\sqrt{r}$); if $\mathcal{A}$

1559 defers, the adversary raises C to U ($\mathcal{A}$ commits at U , OPT paid $\sqrt{LU}$, ratio
1560 $\sqrt{r}$). With $P_i = 0$ the idle term is irrelevant. $\square$

1561 This is a sanity-check rather than a strong result. It says nothing about
1562 why Lechowicz fails empirically, since Lechowicz *achieves* $\sqrt{r}$ in the $\beta = 0$
1563 regime.

1564 9.3. Why the Warm-Pool-Coupled Lower Bound is Hard

1565 A natural attempt is to prove $\text{CR}(\mathcal{A}_{\text{Lech}}) = \Omega(\beta)$ in Variant S, using
1566 a long-running adversarial trace that holds $C = U$ for T_w seconds before
1567 dropping to L . We sketch why this attempt fails in Variant S and what
1568 changes in Variant P.

1569 *Single-container attempt (Variant S) — does not work..* Consider N invo-
1570 cations arriving at $t_k = k\Delta_t$ with $\Delta_t = T_w/N$ and $C(t) = U$ for $t < T_w$,
1571 $C(t) = L$ thereafter.

1572 Under Lechowicz on Variant S: all N invocations defer to $t = T_w$ when
1573 C drops to L , then execute back-to-back warm. The pre- i_1 container's idle
1574 period is at most T_w at $C = U$ (single physical container, by Variant-S defi-
1575 nition). The inter-execution idle periods ($c_k \rightarrow s_{k+1}$ for $k = 1, \dots, N-1$) are
1576 approximately zero (back-to-back execution after T_w). The post-execution
1577 trailing idle is T_w at $C = L$.

$$\text{Cost}_S(\mathcal{A}_{\text{Lech}}) \leq P_i T_w U + N P_a \tau_e L + P_i T_w L.$$

1578 Under OPT on Variant S (commit each at arrival, at $C = U$): inter-arrival
1579 idle periods Δ_t at $C = U$ between consecutive completions, then trailing T_w

1580 at $C = L$.

$$\text{Cost}_S(\text{OPT}) \approx NP_a\tau_e U + (N-1)P_i\Delta_t U + P_iT_w L = N(P_a\tau_e U) + P_iT_w U(1-1/N) + P_iT_w L.$$

1581 The ratio $\text{Cost}_S(\mathcal{A}_{\text{Lech}})/\text{Cost}_S(\text{OPT})$ does *not* grow with N or with β :
 1582 both costs have a $\Theta(P_iT_w \cdot \text{const})$ leading idle term, and the execution costs
 1583 differ by a factor of $r = U/L$ amortized over N invocations. The asymptotic
 1584 ratio (as $N \rightarrow \infty$) is $O(1) + O(1/r)$, trivially close to 1.

1585 *The naive summation $\sum_k P_i(T^* - t_k)U$, which attributes idle cost per-*
 1586 *pending-invocation, over-counts in Variant S: the same physical container is*
 1587 *charged N times for one contiguous idle period.* The construction is therefore
 1588 not a valid lower bound in Variant S.

1589 *Independent-pending attempt (Variant P) — gives the bound..* In Variant P,
 1590 each i_k has its own pending container that idles from t_k to s_k . Under Le-
 1591 chowicz, pending container for i_k idles $(T_w - t_k) = (N - k)\Delta_t$ seconds at
 1592 $C = U$:

$$\text{Cost}_P(\mathcal{A}_{\text{Lech}}) = \sum_{k=1}^N P_i(T_w - k\Delta_t)U + NP_a\tau_e L + NP_iT_w L \approx \frac{P_iT_w UN}{2} + NP_a\tau_e L + NP_iT_w L.$$

1593 Under OPT on Variant P (commit-at-arrival), each container's pre-commit
 1594 idle is zero, post-commit idle T_w :

$$\text{Cost}_P(\text{OPT}) = NP_a\tau_e U + NP_iT_w \bar{C}^{\text{post}}.$$

1595 The amortized per-invocation ratio is

$$\frac{\text{Cost}_P(\mathcal{A}_{\text{Lech}})/N}{\text{Cost}_P(\text{OPT})/N} \rightarrow \frac{P_iT_w U/2 + P_a\tau_e L + P_iT_w L}{P_a\tau_e U + P_iT_w \bar{C}^{\text{post}}}$$

1596 which approaches $\Theta(\beta)$ if $\bar{C}^{\text{post}}$ is small relative to U (i.e., if the post-commit
 1597 window is at low carbon), and a constant otherwise. The bound is much

1598 weaker than the naive $\beta + 1/r$ a per-pending-invocation accounting would
 1599 suggest, and it depends on the adversary’s freedom to choose the post-commit
 1600 carbon trace.

1601 *The simulator’s regime..* The simulator implements neither Variant S nor
 1602 Variant P exactly: warm containers are pooled per (region, function), so
 1603 a pending i_k does not provision a new container if a compatible warm one
 1604 exists. Under heavy load, the pool size grows toward the number of concur-
 1605 rent pending invocations (approaching Variant P); under light load, a single
 1606 warm container serves successive arrivals (approaching Variant S). Lechow-
 1607 icz’s empirical -336% vs FIFO on real data (Table 4) is consistent with
 1608 the intermediate regime: not the trivial Variant S worst case, not the $\Theta(\beta)$
 1609 Variant P worst case.

1610 9.4. Conjecture and Open Problems

1611 We state what we believe but cannot yet prove.

1612 **Conjecture 9.2.** *There exists a multi-container (Variant P-like) cost model*
 1613 *that captures FaaS production behavior, in which any algorithm of the Le-*
 1614 *chowicz form (threshold-based deferral without an idle-energy charge) has*
 1615 *competitive ratio $\Omega(\beta)$ in the regime $\beta \gg \sqrt{r}$.*

1616 Resolving this conjecture requires three pieces of work we have not done:
 1617 (i) a formal multi-container cost model that matches FaaS production se-
 1618 mantics (warm-pool sharing, capacity, function-level isolation), (ii) an ad-
 1619 versary construction in that model that withstands the per-invocation idle
 1620 accounting (not the double-counted Variant S attempt above), and (iii) a
 1621 matching upper-bound algorithm to show the lower bound is tight, ideally a

1622 randomized variant of GREENFAAS’s break-even gate that achieves $O(\sqrt{r})$
1623 on stationary inputs.

1624 We view this as the principal theoretical follow-up the paper suggests.
1625 The empirical evidence (§8.5, Tables 4, 5) is consistent with the conjecture
1626 but does not prove it.

1627 9.5. What This Section Establishes

1628 The theoretical contribution of this paper is honestly modest: *(i)* a for-
1629 malization of the warm-pool-coupled cost model (§9.1) that distinguishes
1630 single-container, independent-pending, and intermediate variants, useful for
1631 follow-up work; *(ii)* the classical one-way trading lower bound (Theorem 9.1)
1632 under the $\beta = 0$ specialization, which is a sanity check rather than a strong
1633 result; *(iii)* a careful working-out of why the natural N-invocation adversarial
1634 construction fails in the single-container model (§9.3), which clarifies what a
1635 future proof would need to handle; *(iv)* an open conjecture (§9.4) that the
1636 warm-pool-coupled regime admits an $\Omega(\beta)$ lower bound on Lechowicz-style
1637 algorithms.

1638 The empirical evidence for the conjecture is strong (Lechowicz empirically
1639 performs worst of all baselines on FaaS workloads, including those where its
1640 theoretical $\sqrt{r}$ guarantee should apply). A complete theoretical resolution is
1641 open work.

1642 10. Production Deployment Architecture

1643 This section addresses an obvious reviewer concern: the headline results
1644 of §8 are produced by a discrete-event simulator (§7), not by a deployed sys-
1645 tem. The validity of conclusions like the Wait-Awhile +305% inflation and

1646 the do-no-harm property rests on the simulator faithfully capturing produc-
1647 tion FaaS dynamics. This section specifies a concrete production-validation
1648 architecture on Knative (the de facto open-source FaaS platform), identifies
1649 the semantic mismatches with our simulator, and proposes mitigations. We
1650 have not deployed this architecture; the discussion is part deployment plan,
1651 part limitations statement.

1652 *10.1. Component Mapping: Simulator to Knative*

1653 *Break-even gate (Lemma 5.1)..* The closed-form break-even rate λ^* that
1654 gates region admission becomes a Kubernetes Custom Resource maintained
1655 by a custom Operator. The Operator periodically (every ~ 60 s) polls Elec-
1656 tricityMaps via the same fetch protocol used in `scripts/fetch_electricitymaps.py`,
1657 queries Prometheus for the arrival-rate estimator $\hat{\lambda}$ produced by the `ArrivalRateTracker`
1658 of Algorithm 1, and publishes a per-region `ConfigMap` indicating whether
1659 the spatial or temporal regime is active. The Operator never makes per-
1660 invocation decisions; it only updates platform state at $\sim$ minute granularity,
1661 leaving per-invocation routing to the proxy below.

1662 *Caveat on poll latency.* A 60-second Operator poll interval means up
1663 to one minute of staleness in the regime classification during rapid carbon-
1664 intensity shifts (e.g., a wind front passing through Germany at midnight).
1665 This produces up to one minute of mis-routing for invocations dispatched
1666 between an intensity change and the next Operator update. The bound
1667 is benign for slow-varying grids (where intensity rarely moves more than 5%
1668 per minute) but worth instrumenting in production: log the Operator update
1669 timestamps to OpenTelemetry alongside the routing decisions, so post-hoc
1670 attribution of mis-routing to poll latency is possible.

1671 *Per-invocation routing.* The natural injection point for per-invocation GREEN-
1672 FAAS decisions is *not* the Kubernetes Scheduler (which routes pods to nodes
1673 within a cluster, not invocations to clusters across regions). Instead, we pro-
1674 pose a sidecar in the Knative `Activator` flow. The `Activator` already sits
1675 between the Knative `Service` ingress and the function pods, mediating cold
1676 starts; adding a GREENFAAS decision pass before the `Activator` forwards the
1677 request is mechanically straightforward.

1678 The proxy reads the `ConfigMap`, computes the per-invocation score from
1679 Algorithm 1, and either (a) holds the request locally (temporal deferral,
1680 $\Delta t \leq$ class-dependent budget) and forwards to the local `Activator` when the
1681 deferral completes, or (b) rewrites the request URL to the alternate cluster’s
1682 `Activator` and forwards. The local `Activator` then handles cold-start detection
1683 and pod-warming as it does today; our policy only changes *which* cluster sees
1684 the request.

1685 *Cross-cluster forwarding.* Cross-cluster forwarding requires that all partici-
1686 pating clusters share a function-image registry and that requests carry suffi-
1687 cient identifying information (function name, request payload). Knative’s ex-
1688 isting multi-tenant model handles this through the `Service.serving.knative.dev/cluster-local`
1689 annotation; for multi-region deployment, each region runs an identically-
1690 named Knative `Service`, and the proxy targets the alternate region’s ingress
1691 URL. No changes to Knative itself are required.

1692 10.2. Replacing the Power Model

1693 The simulator’s $P_a = 4\text{W}$ active, $P_i = 0.3\text{W}$ idle is a calibration to
1694 typical container-level draws on a server-class CPU. Production validation

1695 requires *measured* power, not assumed.

1696 *Kepler.. Kepler* (Kubernetes-based Efficient Power Level Exporter) (Amaral
1697 et al., 2023) uses eBPF to attribute CPU and memory energy consumption
1698 to individual pods on the Kubelet. A pod’s energy draw during execution is
1699 measured at sub-second resolution and published to Prometheus. For warm-
1700 but-idle pods, Kepler measures the residual draw (typically dominated by
1701 background runtime and language-VM overhead rather than the function
1702 code itself), giving an empirical P_i that may differ from our 0.3 W assumption
1703 by a substantial factor depending on the runtime.

1704 The relevance for our paper is that $\beta = P_i T_w / (P_a \tau_e)$ is the dimension-
1705 less quantity governing both the Lemma 5.1 break-even and Conjecture 9.2’s
1706 asymptotic lower bound; production-measured β via Kepler is what deter-
1707 mines the realized empirical behavior. If Kepler-measured β on the chosen
1708 Knative runtime differs from our 150 by a factor of 2–3, the qualitative con-
1709 clusions of §8 should reproduce, but the quantitative magnitudes will shift.

1710 *Kepler measurement caveats..* eBPF-based per-pod power attribution has
1711 known accuracy limitations: published Kepler benchmarks report 5–15%
1712 per-pod attribution error depending on workload type (compute-bound func-
1713 tions show lower error than I/O-bound ones; co-resident pods sharing a CPU
1714 socket are harder to disentangle). This uncertainty propagates directly to
1715 $\beta = P_i T_w / (P_a \tau_e)$: a 10% error on P_i produces a 10% error on β , which
1716 in turn produces a similar-magnitude error on the Lemma 5.1 break-even
1717 rate λ^* . A production deployment should therefore report β and λ^* with
1718 explicit confidence intervals derived from Kepler’s measurement uncertainty,

1719 and verify that the qualitative conclusions of §8 are robust to β varying
1720 within $[100, 200]$.

1721 *Carbon attribution.* Per-pod energy is converted to carbon by multiplying by
1722 the ElectricityMaps-reported $C(t)$ of the cluster’s region at execution time.
1723 This is the same model the simulator uses; the only difference is that the
1724 energy term is measured rather than computed.

1725 10.3. Latency and SLA Instrumentation

1726 The simulator tracks latency as $\text{Lat}_k = (s_k - t_k) + \tau_e + \mathbf{1}[\text{cold}] \tau_c + \text{RTT}(r_k)$.
1727 Production latency includes everything the simulator omits: ingress over-
1728 head, TLS handshake variance, pod scheduling jitter, image-pull delay if the
1729 image is not pre-pulled, language-VM cold-start variance, and network jitter
1730 on cross-region forwards. SLA violations could plausibly come from any of
1731 these sources, not just the GREENFAAS-induced deferral that the simulator
1732 models.

1733 *OpenTelemetry traces.* The proxy sidecar emits distributed traces tagged
1734 with GREENFAAS’s decision (commit time, target region, cold/warm classi-
1735 fication at decision time) and the realized latency components above. The
1736 trace collection backend stores these per-invocation, and the analysis pipeline
1737 computes the SLA violation rate, attributed by component. This separates
1738 “GREENFAAS caused the violation by deferring too long” from “GREENFAAS
1739 forwarded to a region where the Activator had a cold-start spike unrelated
1740 to our policy.”

1741 *10.4. Semantic Mismatch: T_w Granularity*

1742 The most consequential semantic mismatch between our simulator (and
1743 Lemma 5.1) and Knative is the granularity of the warm-pool TTL. The sim-
1744 ulator models T_w as a per-container property: each warm container expires
1745 exactly T_w seconds after its last invocation. Knative’s KPA (Knative Pod Au-
1746 toscaler) maintains T_w as a per-Revision property (`scale-to-zero-grace-period`),
1747 not per-pod, and the scale-to-zero decision is made by a controller poll at
1748 the configured interval, not exactly at the TTL boundary.

1749 The practical consequence: Knative’s effective T_w for a *specific* pod can
1750 range from T_w to $T_w + \delta$ where δ is the controller poll interval (default
1751 60 s). The λ^* computed by Lemma 5.1 is therefore an *approximation* of
1752 the production-realized break-even rate; the approximation error is bounded
1753 by δ/T_w , around 10% for default Knative settings.

1754 *Mitigation..* Two production configurations tighten this: (i) set Knative’s
1755 `container-concurrency: 1` so each warm pod serves one invocation at a time
1756 (matching our single-container model exactly), and (ii) reduce the controller
1757 poll interval to ≤ 5 s. Together these bring the simulator-vs-production T_w
1758 mismatch below 1%.

1759 *Caveat on aggressive polling at scale.* A 5-second poll interval increases
1760 API-server load proportionally; on clusters with >1000 Revision objects, the
1761 load increase can require horizontal scaling of the `kube-apiserver` and add
1762 measurable latency to other control-plane operations. A production deploy-
1763 ment at scale should benchmark the API-server load impact and consider
1764 intermediate values (e.g., 15 s) that retain most of the T_w fidelity while keep-
1765 ing control-plane overhead acceptable. For GREENFAAS’s purposes, even

1766 a 30-second poll interval keeps the T_w mismatch under 5%, which is small
1767 compared to the Kepler-induced uncertainty on β noted in §10.2.

1768 10.5. Cold-Start Variance

1769 The simulator treats τ_c as a constant per-function property (typically 1 s
1770 in our catalog). Knative cold starts have orders of magnitude more variance,
1771 driven by: image-pull time from the container registry (10 ms–30 s depending
1772 on cache state), runtime initialization (sub-second for compiled languages,
1773 multi-second for Python/Node with heavy dependency graphs), and network
1774 setup (typically 100–500 ms).

1775 *Mitigation..* A `DaemonSet` that pre-pulls function images to every node re-
1776 moves the dominant variance source. With pre-pulled images and a fast
1777 runtime (Go or compiled Node), production τ_c converges to 200–500 ms with
1778 low variance, close to our simulator’s 1 s default. The simulator’s τ_c should be
1779 re-calibrated against Kepler-measured cold-start durations once the deploy-
1780 ment is running; if production τ_c differs significantly from 1 s, the $\alpha = \tau_c/\tau_e$
1781 ratio in Lemma 5.1 shifts and λ^* should be recomputed.

1782 10.6. What This Validates and What It Does Not

1783 A production deployment built as above would validate:

- 1784 • Whether the Wait-Awhile and Lechowicz failure modes (§8.5) reproduce
1785 on a real platform with real Kepler-measured energy.
- 1786 • Whether the do-no-harm property (§8.11) holds when production T_w
1787 has the controller-poll jitter described above.
- 1788 • Whether the GREENFAAS-Spatial gap stays within our empirical 0.058pp
1789 ± 0.032 pp range (§8.6) under production conditions.

1790 It would *not* resolve Conjecture 9.2, which requires a formal multi-container
1791 model and an adversary construction; a production deployment can provide
1792 empirical support but not a proof.

1793 The deployment also exposes a class of issues the simulator does not
1794 model: cross-region network partitions, controller failures during a scale-to-
1795 zero event, and registry availability for cold-start image-pulls. These are
1796 infrastructure-reliability issues rather than scheduling-policy issues; we treat
1797 them as orthogonal.

1798 *Why we have not done this..* A full production validation requires multi-
1799 region Kubernetes clusters with Knative installed, Kepler integration, an
1800 ElectricityMaps subscription with sufficient quota, and several days of run-
1801 ning representative workloads. Our build environment does not provide clus-
1802 ter resources, and the validation is a non-trivial engineering project rather
1803 than a paper-revision task. We treat the architecture described above as
1804 a concrete plan for follow-up work, and note it as the most consequential
1805 validation step still remaining.

1806 11. Discussion and Limitations

1807 This section discusses the principal limitations of the work and the direc-
1808 tions we see as most promising for follow-up.

1809 *Single-container warm pools..* Lemma 5.1 is stated for a single warm con-
1810 tainer. Real FaaS platforms maintain warm *pools* of size $n \geq 1$. The multi-
1811 container generalization changes the survival probability $\Pr[X > T_w]$ to a
1812 function of the steady-state distribution of an M/M/ n -style queue, and scales

1813 idle energy with $\mathbb{E}[n - \text{busy}]$. We conjecture an analogous threshold structure
 1814 carries over, but the exact dependence on n involves the (non-closed-form)
 1815 steady-state occupancy distribution and remains future work. The scheduler
 1816 currently applies the single-container lemma to each incremental container
 1817 in the pool, which is the right limit-case behaviour but does not exploit
 1818 joint-pool dynamics. The empirical impact in the regimes we tested is small.

1819 *Reach of the dichotomy on FaaS parameters..* As we acknowledge in §5.2, the
 1820 lemma’s first regime ($\beta \leq \beta_{\text{crit}}$, warm-in-L wins for all $\lambda > 0$) is effectively
 1821 unreachable for realistic FaaS function profiles, where $\beta = P_i T_w / (P_a \tau_e) \in$
 1822 $[50, 200]$ and $\beta_{\text{crit}} = (r - 1)(1 + \alpha) \leq 47$ even for extreme carbon ratios. The
 1823 practical content of the lemma for FaaS scheduling is therefore the second
 1824 regime: the closed-form computation of λ^* that the scheduler uses to gate
 1825 region admission. The dichotomy formulation has theoretical value (it is the
 1826 natural starting point for the proof) and is reachable for adjacent computing
 1827 classes (long-running idle keep-alives, edge containers with short TTLs), but
 1828 the FaaS-specific practical contribution is the λ^* characterization, not the
 1829 classification.

1830 *No demonstrated dominance over pure spatial routing..* A careful read of the
 1831 empirical results in §8.9 reveals that GREENFAAS does not strictly outper-
 1832 form the simpler Spatial baseline on any real-carbon topology we tested. The
 1833 two schedulers stay within 1.3 percentage points across five real-data topolo-
 1834 gies, and Spatial in fact edges GREENFAAS on real ENTSO-E coal-belt data
 1835 (§8.9). GREENFAAS’s distinct value relative to Spatial is therefore not “bet-
 1836 ter carbon reduction” but “topology-robust scheduler that does not require

1837 operator tuning for low-carbon-refuge presence, includes a principled cold-
1838 start trade-off lemma, and preserves do-no-harm across a fine-grained range
1839 of low-opportunity workloads (§8.11)”. We believe this is a more defensible
1840 and practically useful claim than “always wins”. The underlying point is
1841 that on real-world grid carbon data with realistic FaaS arrival patterns, the
1842 spatial axis already captures most of the carbon-aware savings available; the
1843 temporal-shift contribution is necessary for robustness but does not unlock
1844 large additional headroom on top of well-tuned spatial routing.

1845 *Provable online algorithms on FaaS.* Our §8.5 comparison against a FaaS-
1846 adapted port of Lechowicz et al. (2023)’s provably optimal one-way trading
1847 algorithm produced a striking empirical finding: the provably-optimal algo-
1848 rithm performs *worst* of all baselines on FaaS workloads, including worse than
1849 the heuristic Wait-Awhile. The mechanism is the non-separable cost struc-
1850 ture — warm-pool idle energy couples decisions across invocations through
1851 state that Lechowicz’s competitive analysis does not see, so the analytically-
1852 optimal threshold is empirically wrong on FaaS. We have not attempted to
1853 extend the competitive-ratio proof to the warm-pool-coupled cost structure;
1854 the existing framework’s separable-per-job-cost assumption is fundamental
1855 to the analysis and a FaaS-native online algorithm with competitive guaran-
1856 tees over the coupled problem remains the most interesting open theoretical
1857 extension of this work.

1858 *Workload-trace authenticity.* The sensitivity sweeps of §8.13–§8.14 use a
1859 schema-faithful synthetic workload calibrated to the Azure Functions 2019
1860 and 2021 characterizations (Shahrad et al., 2020; Zhang et al., 2021; Mi-
1861 crosoft, 2021), because parameter variation across multiple axes is the pur-

pose of those experiments. The headline and topology experiments (§8.2, §8.6, §8.9) report both the synthetic and the fully-real configurations: real Azure 2021 per-invocation trace (1.98×10^6 invocations over 14 days, $\sim 81\text{k}$ in a 24h slice) paired with real ElectricityMaps carbon-intensity data from the same January 2021 window. Time-aligned fully-real results are in §8.6, Table 5. The qualitative findings of the synthetic experiments survive unmodified on fully-real data; the quantitative magnitudes of the batch-scheduler failure modes are softer on real data (3–5% loss vs FIFO instead of 200–300%), reflecting the real workload’s higher baseline cold-start rate. We note this softening transparently in §8.6.

Non-Poisson arrivals.. The lemma assumes Poisson inter-arrival times, empirically violated by real production workloads (Shahrad et al., 2020). The evaluation on real LWA carbon traces with a schema-faithful bursty workload (§8.3) shows that the scheduler still achieves 64% reduction vs FIFO, within 0.5 percentage points of pure spatial routing; we interpret this as evidence that the lemma’s *qualitative* prescription is robust to non-Poisson arrivals even though the *exact* break-even rate λ^* shifts. A formal extension to general renewal processes is open.

PUE heterogeneity.. The lemma assumes similar PUE across regions. Real data centres span a non-trivial range (PUE 1.1 in hyperscale Nordic sites; PUE 1.5 in older facilities). The lemma extends by replacing $r = C_H/C_L$ with $r' = (C_H \cdot \text{PUE}_H)/(C_L \cdot \text{PUE}_L)$, but our evaluation uses a uniform $\text{PUE} = 1.2$, close to the Uptime Institute’s reported global-average data-centre PUE in recent annual surveys (which has plateaued near 1.5 industry-wide but is

1886 closer to 1.1–1.2 for modern hyperscale facilities of the kind that run multi-
 1887 region FaaS). A PUE-heterogeneous evaluation with measured per-site values
 1888 is a natural follow-up.

1889 *Robustness to power-model assumptions..* Our simulator uses $P_a = 4\text{ W}$ ac-
 1890 tive and $P_i = 0.3\text{ W}$ idle as defaults, calibrated to typical container-level
 1891 draws. Because production-measured power (via Kepler, §10.2) may differ,
 1892 we verified that the scheduler ranking is invariant to the power model. Scaling
 1893 P_a and P_i by factors in $\{0.5, 1.0, 1.5\}$, including the asymmetric perturba-
 1894 tions $(P_a \times 0.5, P_i \times 1.5)$ and $(P_a \times 1.5, P_i \times 0.5)$, leaves the ordering Spatial <
 1895 GREENFAAS < GreenFaaS-v1 < FIFO < Wait-Awhile unchanged in every
 1896 case (real Azure 2021 + real EM Jan 2021, 5-region topology). The abso-
 1897 lute carbon scales with the power assumptions, as expected, but the relative
 1898 scheduler performance — the paper’s actual claim — does not depend on the
 1899 specific P_a/P_i values.

1900 *Cold-start-time sensitivity..* The simulator treats the cold-start duration τ_c
 1901 as a constant per-function property ($\sim 1\text{ s}$ in our catalog). In production, τ_c
 1902 varies with language runtime and image-pull state, ranging from sub-second
 1903 to several seconds, and concurrent cold starts can contend for I/O. Since
 1904 $\alpha = \tau_c/\tau_e$ enters the Lemma 5.1 threshold, larger τ_c raises β_{crit} and shifts
 1905 λ^* , making the gate *more* willing to keep a warm container. We did not
 1906 sweep τ_c explicitly; a 0.5–5 s span (a $10\times$ range) changes λ^* monotonically
 1907 but does not reverse the qualitative gate decisions for our workload’s arrival
 1908 rates, because the spatial carbon ratio dominates the gate in the multi-region
 1909 topologies where savings occur.

1910 *Mean-carbon estimation transient..* The idle-energy correction that enables
 1911 do-no-harm uses $\bar{C}_r$, the long-run mean carbon intensity per region. During
 1912 the first hours of a fresh deployment a running estimate of $\bar{C}_r$ may be biased,
 1913 weakening the correction and potentially admitting an unprofitable deferral.
 1914 A production deployment should seed $\bar{C}_r$ with a historical annual mean (e.g.
 1915 from ElectricityMaps) rather than a cold running average, and log per-region
 1916 $\bar{C}_r$ estimates alongside deferral counts for post-hoc auditing.

1917 *No competitive-ratio bound..* As discussed in §4.2, we make no claim of com-
 1918 petitive optimality. The non-separable cost structure (idle energy couples
 1919 decisions across invocations through the warm-pool state) together with the
 1920 adversarial-perturbation argument rule out a finite competitive ratio in the
 1921 worst case. The Lechowicz et al. double-threshold framework (Lechowicz
 1922 et al., 2023) gives optimal ratios for a strictly simpler model and is a natu-
 1923 ral starting point for a future theoretical extension. The principal obstacle
 1924 is the warm-pool idle-energy term, which has no analogue in pause/resume.
 1925 We view this as the most interesting open theoretical question raised by the
 1926 paper.

1927 *Embodied carbon..* GREENFAAS optimizes *operational* carbon and ignores
 1928 *embodied* carbon. EcoLife (Jiang et al., 2024) addresses embodied carbon by
 1929 routing functions to older, already-amortised hardware. This is an orthogo-
 1930 nal axis to the spatial/temporal grid- carbon axes we exploit; a natural com-
 1931 position would route an invocation first by GREENFAAS’s spatial/temporal
 1932 logic to a region/time, and then within that region by EcoLife’s hardware-
 1933 generation logic. We have not built this composition; it is the most concrete
 1934 next-paper-sized direction this work suggests.

1935 *Real-data validation..* Both axes of our evaluation are validated on real data.
1936 The grid-carbon side uses real ENTSO-E / CAISO 2020 data for four re-
1937 gions (§8.3, §8.9) and real ElectricityMaps January 2021 data for five regions
1938 (§8.6). The workload side is validated on the real Azure Functions 2021
1939 per-invocation trace (1.98×10^6 invocations over 14 days), paired with time-
1940 aligned ElectricityMaps carbon data from the same January 2021 window, in
1941 the fully-real experiments of §8.6. The controlled sensitivity sweeps (§8.10–
1942 §8.14) use a schema-faithful synthetic workload because parameter variation
1943 across multiple axes is their purpose, but the headline ordering is confirmed
1944 on the fully-real data: the Wait-Awhile and Lechowicz failure modes, the
1945 GREENFAAS/Spatial near-match, the v1 ablation gap, and the do-no-harm
1946 property all reproduce, with the batch-scheduler failure magnitudes softer on
1947 real data (3–5% vs FIFO) than on synthetic (up to -306%), reflecting the
1948 real workload’s higher baseline cold-start rate.

1949 *Real-trace scope and LLM workloads..* The Azure traces date from 2019 and
1950 2021. The serverless mix has evolved since then, most notably with LLM
1951 inference functions, which have substantially longer durations and higher
1952 per-invocation energy. Our latency-class heuristics would classify LLM infer-
1953 ence as background, which is technically correct under our definition but un-
1954 derstates the schedulable opportunity for interactive LLM applications such
1955 as chatbots. We expect the qualitative findings to carry over — GREEN-
1956 FAAS’s do-no-harm property is workload-independent — but absolute num-
1957 bers would shift on a contemporary LLM-heavy trace.

1958 *Production-system validation..* GREENFAAS is evaluated entirely in simula-
1959 tion. The Lemma 5.1 break-even computation is cheap enough ($< 100 \mu\text{s}$ per

call) that integration into a production scheduler such as Knative or AWS Lambda is plausible; the closest production point of comparison, Green-Courier (Chadha et al., 2023), is a Kubernetes plugin with ~ 24 ms binding-latency overhead above the default scheduler. A multi-region Knative deployment with measured grid carbon would be the strongest possible confirmation of the simulator results.

Summary. We interpret these limitations not as caveats that weaken the contributions but as *scope statements* that delimit what we have shown. The closed-form trade-off characterization, the SLA-tiered algorithm, the do-no-harm empirical property, and the forecast-invariance result are robust within the stated assumptions. The interesting next steps are the multi-container generalization, the embodied-carbon composition with EcoLife, and the production-system validation — each a tractable follow-up rather than a fundamental obstacle.

12. Conclusion

We have presented GREENFAAS, a carbon-aware scheduling framework for serverless workloads. The technical core is Lemma 5.1, a closed-form characterization of the cold-start carbon trade-off, and the recognition (§5.4) that the same dimensionless idle-cost ratio governs the analogous temporal-shift decision. The resulting SLA-tiered algorithm has $O(R \cdot N)$ per-invocation cost and no cross-invocation coordination beyond an arrival-rate estimator.

On real LWA 2020 carbon traces, GREENFAAS reduces operational carbon by 64–83% on multi-region topologies with a low-carbon refuge, by 52–53% on coal-belt topologies with limited spatial opportunity, and by 0%

(matching FIFO by design) on single-region topologies where no carbon-aware opportunity exists. GREENFAAS matches pure spatial routing within 1.3 percentage points across all real-carbon topologies; it does not demonstrate strict dominance over Spatial on any real-data configuration we tested. GREENFAAS’s distinct contribution relative to Spatial is *topology-robustness* (matching Spatial when Spatial works, falling back to FIFO byte-for-byte when it does not) and the principled cold-start trade-off underlying its decisions, not a larger carbon reduction.

Three open directions stand out for follow-up: the multi-container generalization of Lemma 5.1 (§11), a FaaS-adapted comparison against the Lechowicz et al. (2023) double-threshold algorithm, and a production-system validation on Knative. The simulator, trace loaders, scheduler implementations, Caribou port, and manuscript sources are released as open-source artifacts under the Apache-2.0 license upon publication, to support all three.

References

- M. Chadha, T. Subramanian, E. Arima, M. Gerndt, M. Schulz, O. Abboud, Greencourier: Carbon-aware scheduling for serverless functions, in: Proceedings of the 9th International Workshop on Serverless Computing, WoSC ’23, Association for Computing Machinery, New York, NY, USA, 2023, p. 18–23. URL: <https://doi.org/10.1145/3631295.3631396>. doi:10.1145/3631295.3631396.
- V. U. Gsteiger, P. H. D. Long, Y. J. Sun, P. Javanrood, M. Shahradd, Caribou: Fine-grained geospatial shifting of serverless applications for sustainability, in: Proceedings of the ACM SIGOPS 30th Symposium

on Operating Systems Principles, SOSP '24, Association for Computing Machinery, New York, NY, USA, 2024, p. 403–420. URL: <https://doi.org/10.1145/3694715.3695954>. doi:10.1145/3694715.3695954.

Y. Jiang, R. B. Roy, B. Li, D. Tiwari, Ecolife: Carbon-aware serverless function scheduling for sustainable computing, in: SC24: International Conference for High Performance Computing, Networking, Storage and Analysis, 2024, pp. 1–15. doi:10.1109/SC41406.2024.00018.

S. Qi, H. Moore, N. Hogade, D. Milojicic, C. Bash, S. Pasricha, Casa: A framework for slo- and carbon-aware autoscaling and scheduling in serverless cloud computing, in: 2024 IEEE 15th International Green and Sustainable Computing Conference (IGSC), 2024, pp. 1–6. doi:10.1109/IGSC64514.2024.00010.

P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, L. Thamsen, Let’s wait awhile: how temporal workload shifting can reduce carbon emissions in the cloud, in: Proceedings of the 22nd International Middleware Conference, Middleware ’21, Association for Computing Machinery, New York, NY, USA, 2021, p. 260–272. URL: <https://doi.org/10.1145/3464298.3493399>. doi:10.1145/3464298.3493399.

dos-group, Let’s wait awhile: Carbon-intensity dataset, <https://github.com/dos-group/lets-wait-awhile>, 2021. Accessed 2026.

A. Lechowicz, N. Christianson, J. Zuo, N. Bashir, M. Hajiesmaili, A. Wierman, P. Shenoy, The online pause and resume problem: Optimal algorithms and an application to carbon-aware load shifting, Proc. ACM Meas.

2031 Anal. Comput. Syst. 7 (2023). URL: <https://doi.org/10.1145/3626776>.
 2032 doi:10.1145/3626776.

2033 T. Sukprasert, A. Souza, N. Bashir, D. Irwin, P. Shenoy, On the limita-
 2034 tions of carbon-aware temporal and spatial workload shifting in the cloud,
 2035 in: Proceedings of the 19th European Conference on Computer Systems
 2036 (EuroSys), 2024.

2037 A. Radovanović, R. Koningstein, I. Schneider, B. Chen, A. Duarte, B. Roy,
 2038 D. Xiao, M. Haridasan, P. Hung, N. Care, S. Talukdar, E. Mullen,
 2039 K. Smith, M. Cottman, W. Cirne, Carbon-aware computing for dat-
 2040 acenters, IEEE Transactions on Power Systems 38 (2023) 1270–1280.
 2041 doi:10.1109/TPWRS.2022.3173250.

2042 W. A. Hanafy, Q. Liang, N. Bashir, D. Irwin, P. Shenoy, Carbonscaler:
 2043 Leveraging cloud workload elasticity for optimizing carbon-efficiency, Proc.
 2044 ACM Meas. Anal. Comput. Syst. 7 (2023). URL: [https://doi.org/10.](https://doi.org/10.1145/3626788)
 2045 [1145/3626788](https://doi.org/10.1145/3626788). doi:10.1145/3626788.

2046 Edgerun, faas-sim: A faas simulator for trace-driven workloads, [https://](https://github.com/edgerun/faas-sim)
 2047 github.com/edgerun/faas-sim, 2022. Accessed 2026.

2048 dos-group, vessim: A co-simulation framework for carbon-aware computing,
 2049 <https://github.com/dos-group/vessim>, 2023. Accessed 2026.

2050 Y. Zhang, I. n. Goiri, G. I. Chaudhry, R. Fonseca, S. Elnikety, C. Delim-
 2051 itrou, R. Bianchini, Faster and cheaper serverless computing on har-
 2052 vested resources, in: Proceedings of the ACM SIGOPS 28th Sympos-
 2053 ium on Operating Systems Principles, SOSP '21, Association for Com-

2054 putting Machinery, New York, NY, USA, 2021, p. 724–739. URL: <https://doi.org/10.1145/3477132.3483580>. doi:10.1145/3477132.3483580.

2055

2056 Microsoft, Azure public dataset, [https://github.com/Azure/](https://github.com/Azure/AzurePublicDataset)

2057 [AzurePublicDataset](https://github.com/Azure/AzurePublicDataset), 2021. Accessed 2026.

2058 A. Souza, N. Bashir, J. Murillo, W. Hanafy, Q. Liang, D. Irwin, P. Shenoy,

2059 Ecovisor: A virtual energy system for carbon-efficient applications, in:

2060 Proceedings of the 28th ACM International Conference on Architectural

2061 Support for Programming Languages and Operating Systems, Volume

2062 2, ASPLOS 2023, Association for Computing Machinery, New York,

2063 NY, USA, 2023, p. 252–265. URL: [https://doi.org/10.1145/3575693.](https://doi.org/10.1145/3575693.3575709)

2064 [3575709](https://doi.org/10.1145/3575693.3575709). doi:10.1145/3575693.3575709.

2065 R. Bostandoost, A. Lechowicz, W. A. Hanafy, N. Bashir, P. Shenoy, M. Ha-

2066 jiesmaili, Lacs: Learning-augmented algorithms for carbon-aware resource

2067 scaling with uncertain demand, in: Proceedings of the 15th ACM Inter-

2068 national Conference on Future and Sustainable Energy Systems, e-Energy

2069 '24, Association for Computing Machinery, New York, NY, USA, 2024,

2070 p. 27–45. URL: <https://doi.org/10.1145/3632775.3661942>. doi:10.

2071 [1145/3632775.3661942](https://doi.org/10.1145/3632775.3661942).

2072 L. Lin, A. A. Chien, Adapting datacenter capacity for greener datacen-

2073 ters and grid, in: Proceedings of the 14th ACM International Confer-

2074 ence on Future Energy Systems, e-Energy '23, Association for Comput-

2075 ing Machinery, New York, NY, USA, 2023, p. 200–213. URL: <https://doi.org/10.1145/3575813.3595197>.

2076 [doi:10.1145/3575813.3595197](https://doi.org/10.1145/3575813.3595197).

2077 S. Qi, H. Moore, N. Hogade, D. Milojicic, C. Bash, S. Pasricha, A frame-
2078 work for slo, carbon, and wastewater-aware sustainable faas cloud platform
2079 management, in: 2024 IEEE 15th International Green and Sustainable
2080 Computing Conference (IGSC), 2024, pp. 35–36. doi:10.1109/IGSC64514.
2081 2024.00015.

2082 J. Serenari, S. Sreekumar, K. Zhao, S. Sarkar, S. Lee, Greenwhisk: Emission-
2083 aware computing for serverless platform, in: 2024 IEEE International
2084 Conference on Cloud Engineering (IC2E), 2024, pp. 44–54. doi:10.1109/
2085 IC2E61754.2024.00012.

2086 A. Souza, S. Jasoria, B. Chakrabarty, A. Bridgwater, A. Lundberg, F. Skogh,
2087 A. Ali-Eldin, D. Irwin, P. Shenoy, Casper: Carbon-aware scheduling
2088 and provisioning for distributed web services, in: Proceedings of the
2089 14th International Green and Sustainable Computing Conference, IGSC
2090 '23, Association for Computing Machinery, New York, NY, USA, 2024,
2091 p. 67–73. URL: <https://doi.org/10.1145/3634769.3634812>. doi:10.
2092 1145/3634769.3634812.

2093 M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke,
2094 E. Laureano, C. Tresness, M. Russinovich, R. Bianchini, Serverless
2095 in the wild: Characterizing and optimizing the serverless workload at
2096 a large cloud provider, in: 2020 USENIX Annual Technical Confer-
2097 ence (USENIX ATC 20), USENIX Association, 2020, pp. 205–218. URL:
2098 <https://www.usenix.org/conference/atc20/presentation/shahrad>.

2099 D. Du, T. Yu, Y. Xia, B. Zang, G. Yan, C. Qin, Q. Wu, H. Chen, Catalyzer:
2100 Sub-millisecond startup for serverless computing with initialization-less

booting, in: Proceedings of the Twenty-Fifth International Conference
on Architectural Support for Programming Languages and Operating Sys-
tems, ASPLOS '20, Association for Computing Machinery, New York,
NY, USA, 2020, p. 467–481. URL: <https://doi.org/10.1145/3373376.3378512>. doi:10.1145/3373376.3378512.

A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Pi-
wonka, D.-M. Popa, Firecracker: Lightweight virtualization for serverless
applications, in: 17th USENIX Symposium on Networked Systems De-
sign and Implementation (NSDI 20), USENIX Association, Santa Clara,
CA, 2020, pp. 419–434. URL: [https://www.usenix.org/conference/
nsdi20/presentation/agache](https://www.usenix.org/conference/nsdi20/presentation/agache).

A. Fuerst, P. Sharma, Faas-cache: keeping serverless computing alive with
greedy-dual caching, in: Proceedings of the 26th ACM International Con-
ference on Architectural Support for Programming Languages and Oper-
ating Systems, ASPLOS '21, Association for Computing Machinery, New
York, NY, USA, 2021, p. 386–400. URL: [https://doi.org/10.1145/
3445814.3446757](https://doi.org/10.1145/3445814.3446757). doi:10.1145/3445814.3446757.

P. Sharma, A. Fuerst, Accountable carbon footprints and energy profiling
for serverless functions, in: Proceedings of the 2024 ACM Symposium
on Cloud Computing, SoCC '24, Association for Computing Machinery,
New York, NY, USA, 2024, p. 522–541. URL: [https://doi.org/10.1145/
3698038.3698531](https://doi.org/10.1145/3698038.3698531). doi:10.1145/3698038.3698531.

R. El-Yaniv, A. Fiat, R. M. Karp, G. Turpin, Optimal search and one-way
trading online algorithms, *Algorithmica* 30 (2001) 101–139.

2125 M. Amaral, H. Chen, T. Chiba, R. Nakazawa, S. Choochotkaew, E. K. Lee,
2126 T. Eilam, Kepler: A framework to calculate the energy consumption of con-
2127 tainerized applications, in: 2023 IEEE 16th International Conference on
2128 Cloud Computing (CLOUD), 2023, pp. 69–71. doi:10.1109/CLOUD60044.
2129 2023.00017.

Algorithm 3 SCORESLOTS($f, h, \mathcal{R}^*, \Delta^*, \hat{C}, \mathcal{S}, t_0$)

```
1:  $(r^*, s^*, S^*) \leftarrow (\perp, \perp, +\infty)$ 
2: for each  $r \in \mathcal{R}^*$  do
3:    $\delta \leftarrow$  forecast step;  $N \leftarrow \lceil \Delta^* / \delta \rceil + 1$ 
4:    $\bar{C}_r \leftarrow$  long-run mean of  $C_r$ 
5:    $w_0 \leftarrow \mathbf{1}[W_{r,f}(t_0) > 0]$ 
6:   for  $k = 0, 1, \dots, N - 1$  do
7:      $t_k \leftarrow t_0 + k\delta + \text{jitter}(f, i, k) \cdot \delta$ 
8:      $\text{cold} \leftarrow \neg(w_0 \wedge k = 0)$ 
9:      $\text{eta} \leftarrow (t_k - t_0) + \rho(h, r) + \text{cold} \cdot \tau_c(f) + \tau_e(f)$ 
10:    if  $\text{eta} > D(f)$  then continue
11:    end if
12:     $C_k \leftarrow \hat{C}_r(t_k)$ 
13:     $S \leftarrow E_{\text{exec}}(f, r)C_k + \text{cold} \cdot E_{\text{cs}}(f, r)C_k + E_{\text{idle}}(f, r, t_k - t_0)\bar{C}_r$ 
14:    if  $S < S^*$  then  $(r^*, s^*, S^*) \leftarrow (r, t_k, S)$ 
15:    end if
16:  end for
17: end for
18: if  $r^* = \perp$  then return  $(h, t_0)$ 
19: end if
20: return  $(r^*, s^*)$ 
```
